香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 18

Memory:
caching wrap-up

Prof. Yunming XIAO

School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Page replacement

Page replacement policies

- Why do we care about replacement policy?
  - Replacement is an issue with any cache
  - Particularly important with pages
    - The cost of being wrong is high: must go to disk
    - Must keep important pages in memory, not toss them out

Page replacement policies

- FIFO (First In, First Out)
  - Throw out oldest page. Be fair – let every page live in memory for same amount of time.
  - Bad – throws out heavily used pages instead of infrequently used
- RANDOM:
  - Pick random page for every replacement
  - Typical solution for TLB's. Simple hardware
  - Pretty unpredictable – makes it hard to make real-time guarantees
- MIN (Minimum):
  - Replace page that won't be used for the longest time
  - Great (provably optimal), but can't really know future…
  - But past is a good predictor of the future …

Example: FIFO (strawman)

- Suppose we have 3 page frames, 4 virtual pages, and following reference stream:
  - A B C A B D A D B C B
- Consider FIFO Page replacement:

Ref: Page:	A	B	C	A	B	D	A	D	B	C	B
1	A					D				C	
2		B					A				
3			C						B		

- FIFO: 7 faults
- When referencing D, replacing A is bad choice, since need A again right away

Example: MIN / LRU

- Suppose we have the same reference stream:
  - A B C A B D A D B C B
- Consider MIN Page replacement:

Ref: Page:	A	B	C	A	B	D	A	D	B	C	B
1	A									C	
2		B									
3			C			D					

- MIN: 5 faults
  - Where will D be brought in? Look for page not referenced farthest in future
- What will LRU do?
  - Same decisions as MIN here, but won't always be true!

Is LRU guaranteed to perform well?

- Consider the following: A B C D A B C D A B C D
- LRU Performs as follows (same as FIFO here):

Ref: / Page:	A	B	C	D	A	B	C	D	A	B	C	D
1	A			D			C			B		
2		B			A			D			C	
3			C			B			A			D

  - Every reference is a page fault!
- Fairly contrived example of working set of N+1 on N frames

When will LRU perform badly?

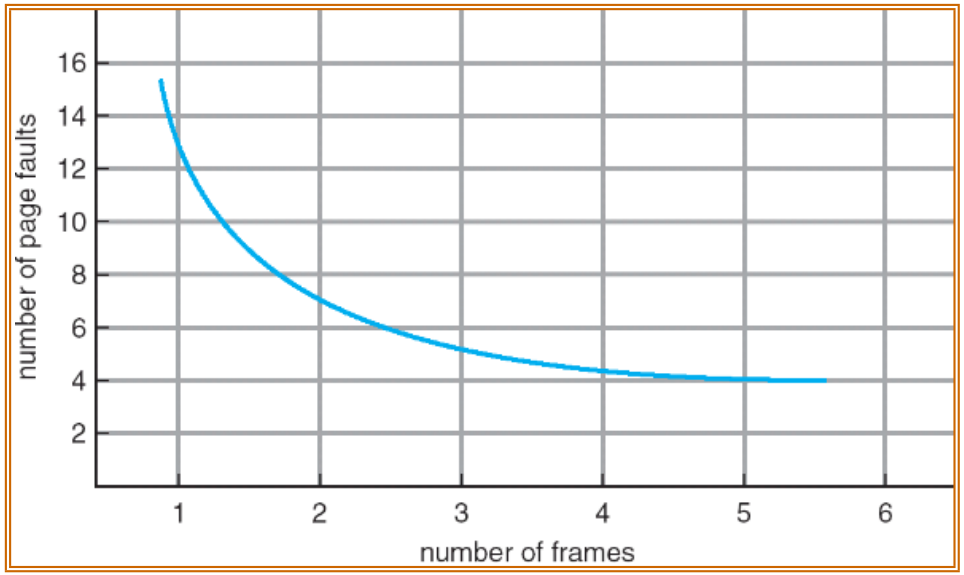
- Consider the following: A B C D A B C D A B C D
- LRU Performs as follows (same as FIFO here):

Ref: / Page:	A	B	C	D	A	B	C	D	A	B	C	D
1	A			D			C			B		
2		B			A			D			C	
3			C			B			A			D

  - Every reference is a page fault!
- MIN Does much better:

Ref: / Page:	A	B	C	D	A	B	C	D	A	B	C	D
1	A									B		
2		B					C					
3			C	D								

Graph of page faults versus the number of frames



- One desirable property: When you add memory the miss rate drops (stack property)
  - Does this always happen?
  - Seems like it should, right?
- No: Bélády's anomaly
  - Certain replacement algorithms (FIFO) don't have this obvious property!

Adding memory doesn't always help fault rate

- Does adding memory reduce number of page faults?
  - Yes for LRU and MIN
  - Not necessarily for FIFO! (Called Bélády's anomaly)

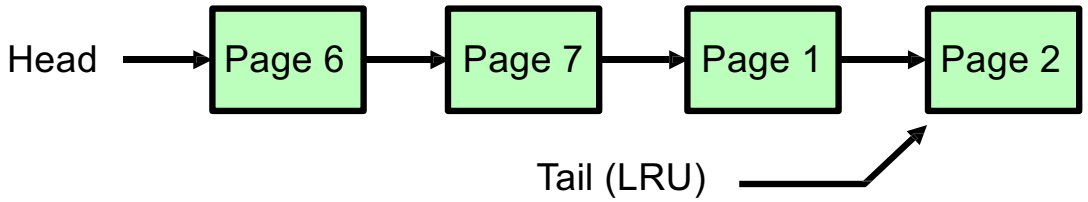
Ref: Page:	A	B	C	D	A	B	E	A	B	C	D	E
1	A			D			E					
2		B			A					C		
3			C			B					D	

Ref: Page:	A	B	C	D	A	B	E	A	B	C	D	E
1	A						E				D	
2		B						A				E
3			C						B			
4				D						C		

- After adding memory:
  - With FIFO, contents can be completely different
  - In contrast, with LRU or MIN, contents of memory with X pages are a subset of contents with X+1 Page

LRU implementation

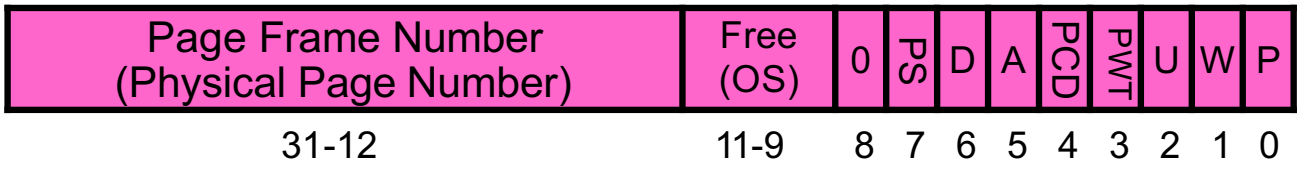
- How to implement LRU? Use a list:



  - On each use, remove page from list and place at head
  - LRU page is at tail
- Problems with this scheme for paging?
  - Need to know immediately when page used so that can change position in list…
  - Many instructions for each hardware access
- In practice, people approximate LRU (more later)

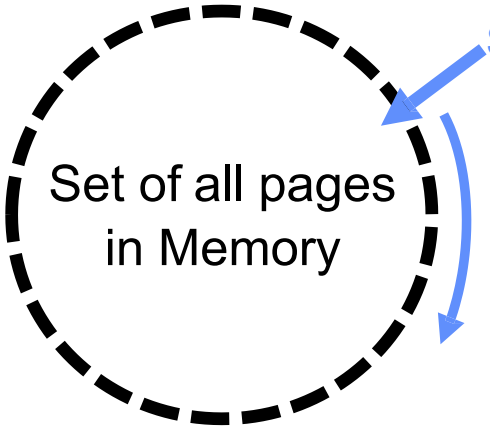
Approximating LRU: recall PTE bits

- Which bits of a PTE entry can help us approximate LRU?
- Remember Intel PTE:



  - The "Present" bit (called "Valid" elsewhere):
    - P==0: Page is invalid and a reference will cause page fault
    - P==1: Page frame number is valid and MMU is allowed to proceed with translation
  - The "Writable" bit (could have opposite sense and be called "Read-only"):
    - W==0: Page is read-only and cannot be written.
    - W==1: Page can be written
  - The "Accessed" bit (called "Use" elsewhere):
    - A==0: Page has not been accessed (or used) since last time software set A→0
    - A==1: Page has been accessed (or used) since last time software set A→0
  - The "Dirty" bit (called "Modified" elsewhere):
    - D==0: Page has not been modified (written) since PTE was loaded
    - D==1: Page has changed since PTE was loaded

Approximating LRU: clock algorithm

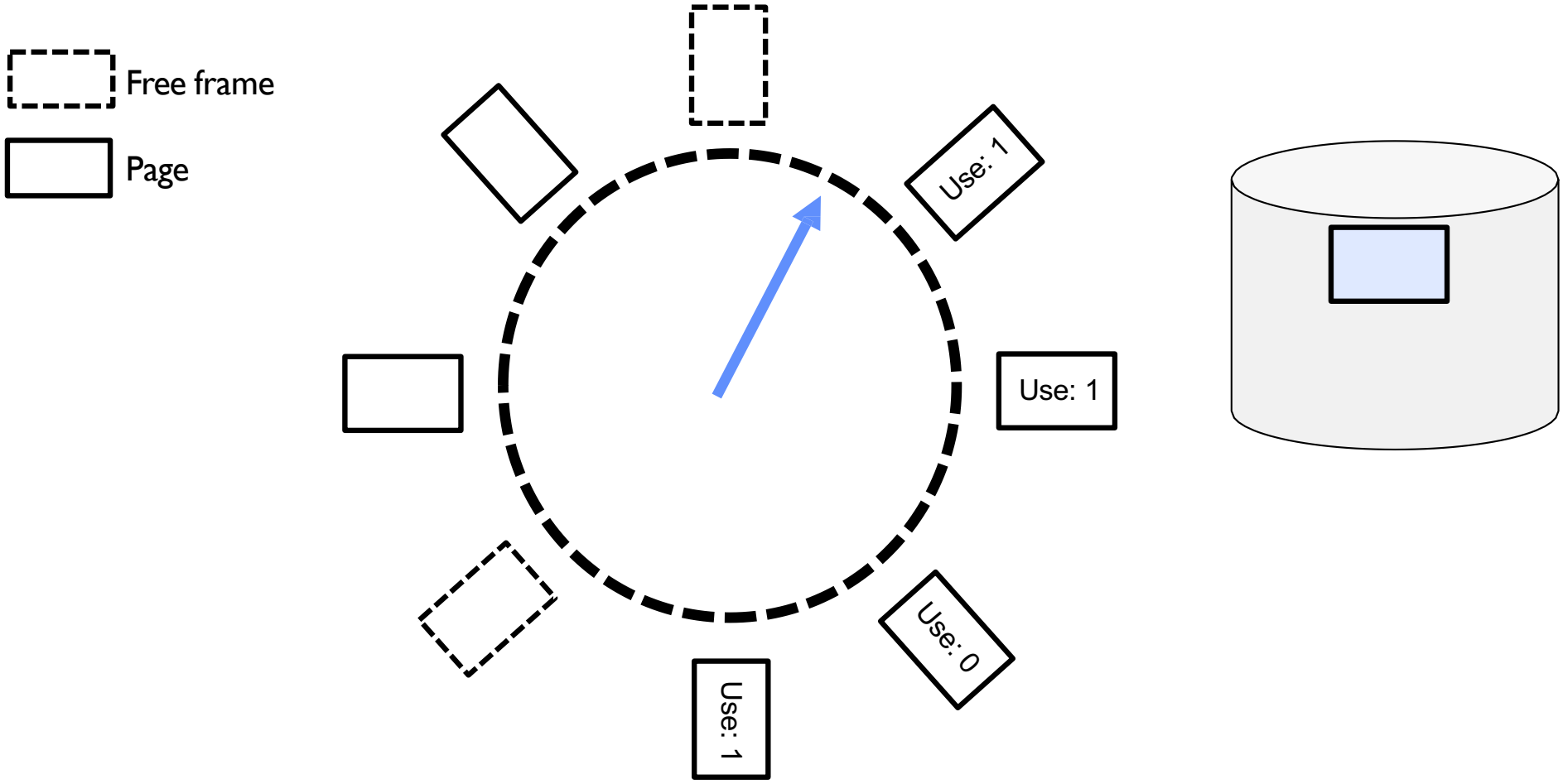


Single Clock Hand:

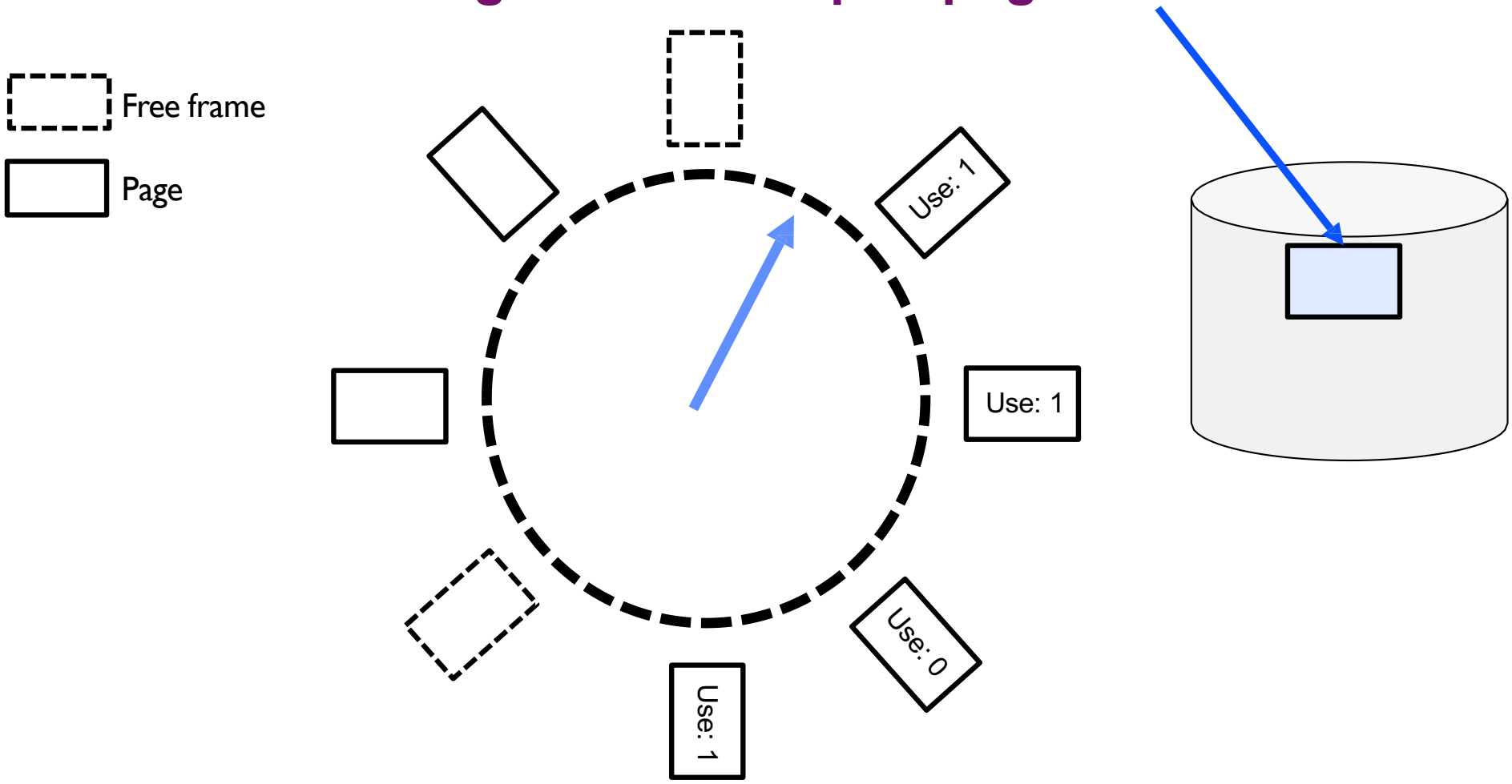
Advances only on page fault!
Check for pages not used recently
Mark pages as not used recently

- Clock Algorithm: Arrange physical pages in circle with single clock hand
  - Approximate LRU (approximation to approximation to MIN)
  - Replace an old page, not the oldest page
- Details:
  - Hardware "use" bit per physical page (called "accessed" in Intel architecture):
    - Hardware sets use bit on each reference
    - If use bit isn't set, means not referenced in a long time
    - Some hardware sets use bit in the TLB; must be copied back to PTE when TLB entry gets replaced
  - On page fault:
    - Advance clock hand (not real time)
    - Check use bit: 1→ used recently; clear and leave alone
      0→ selected candidate for replacement

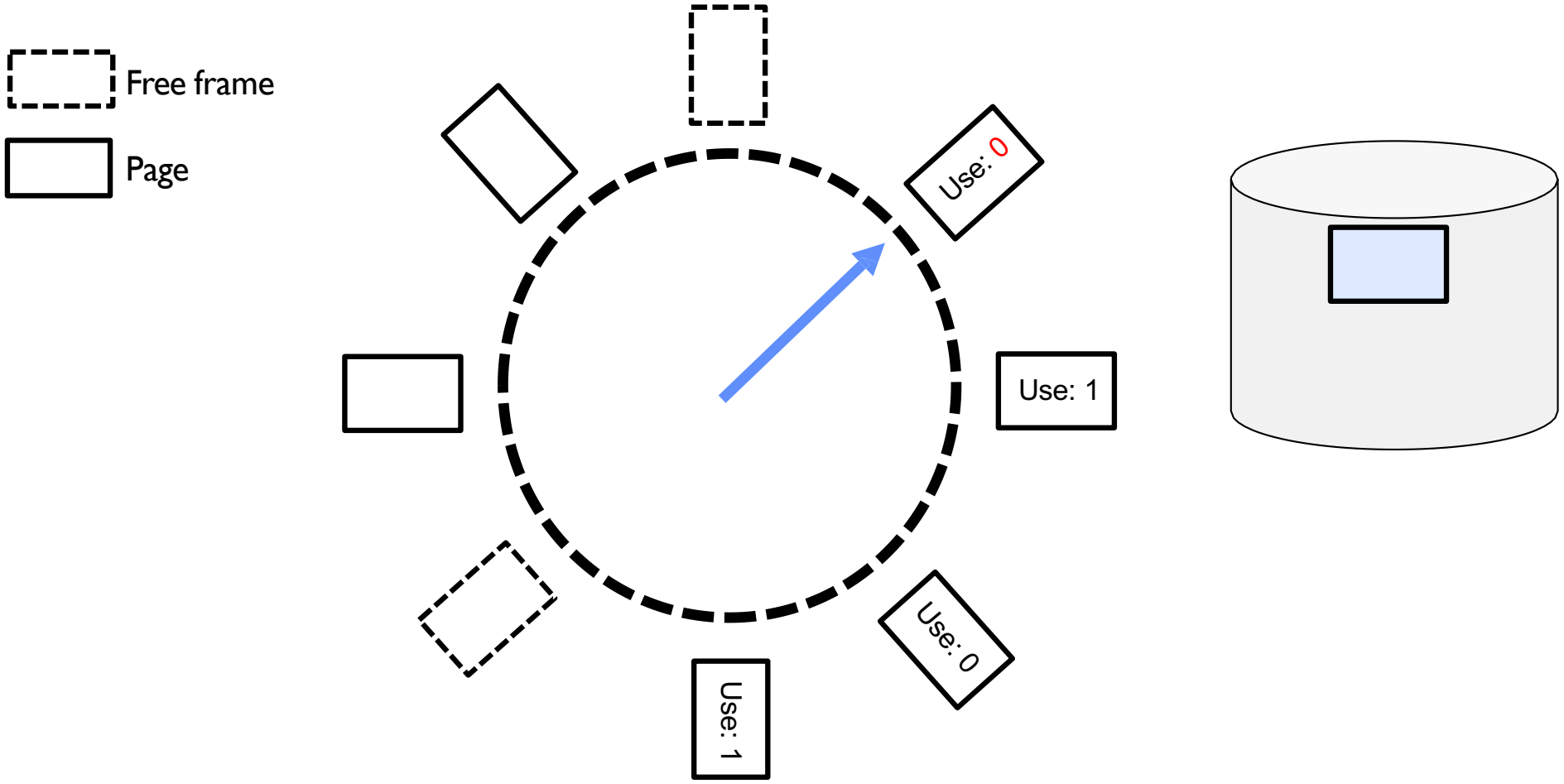
Clock algorithm example



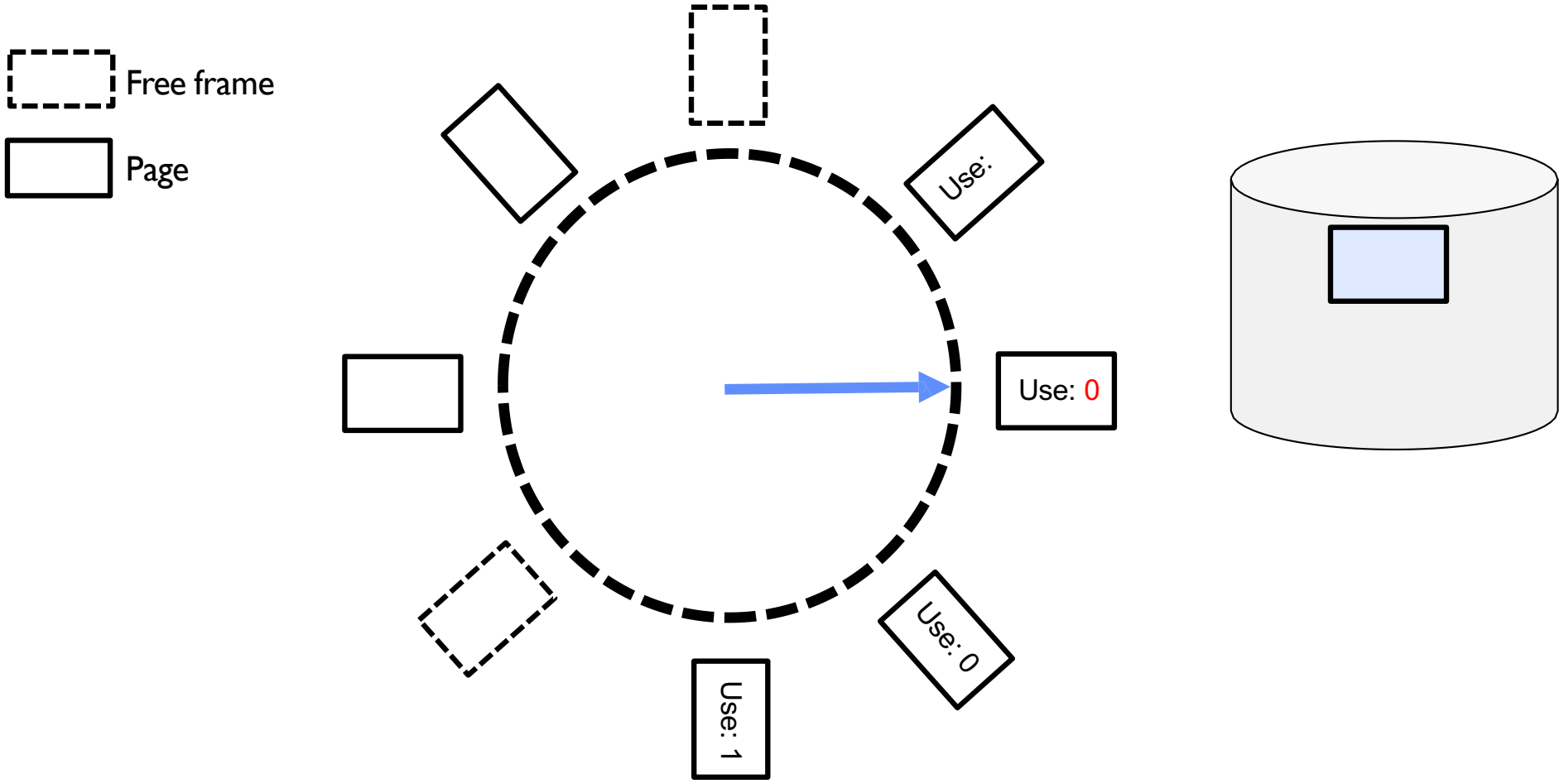
Clock algorithm example: page fault



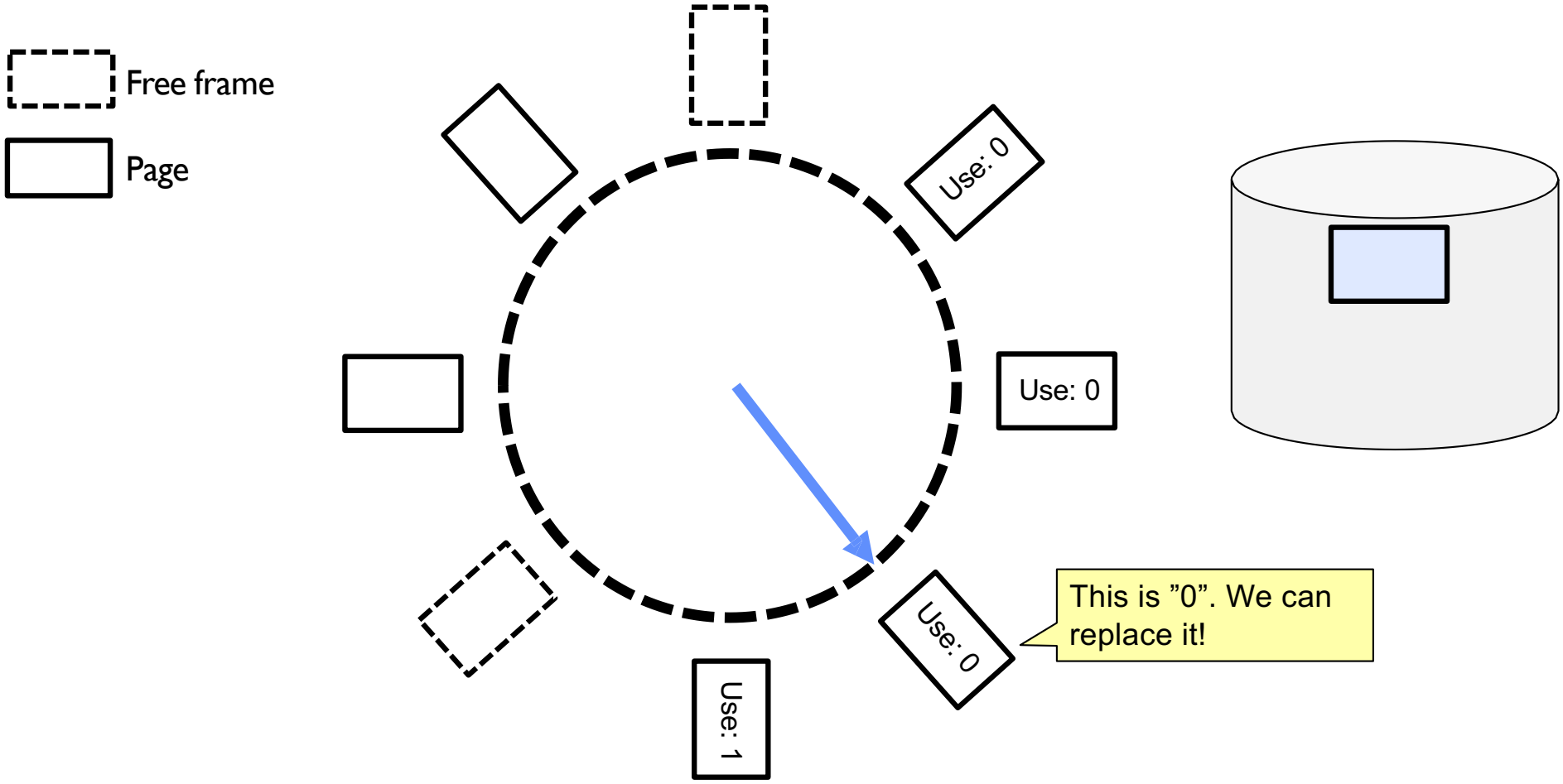
Clock algorithm example: page fault



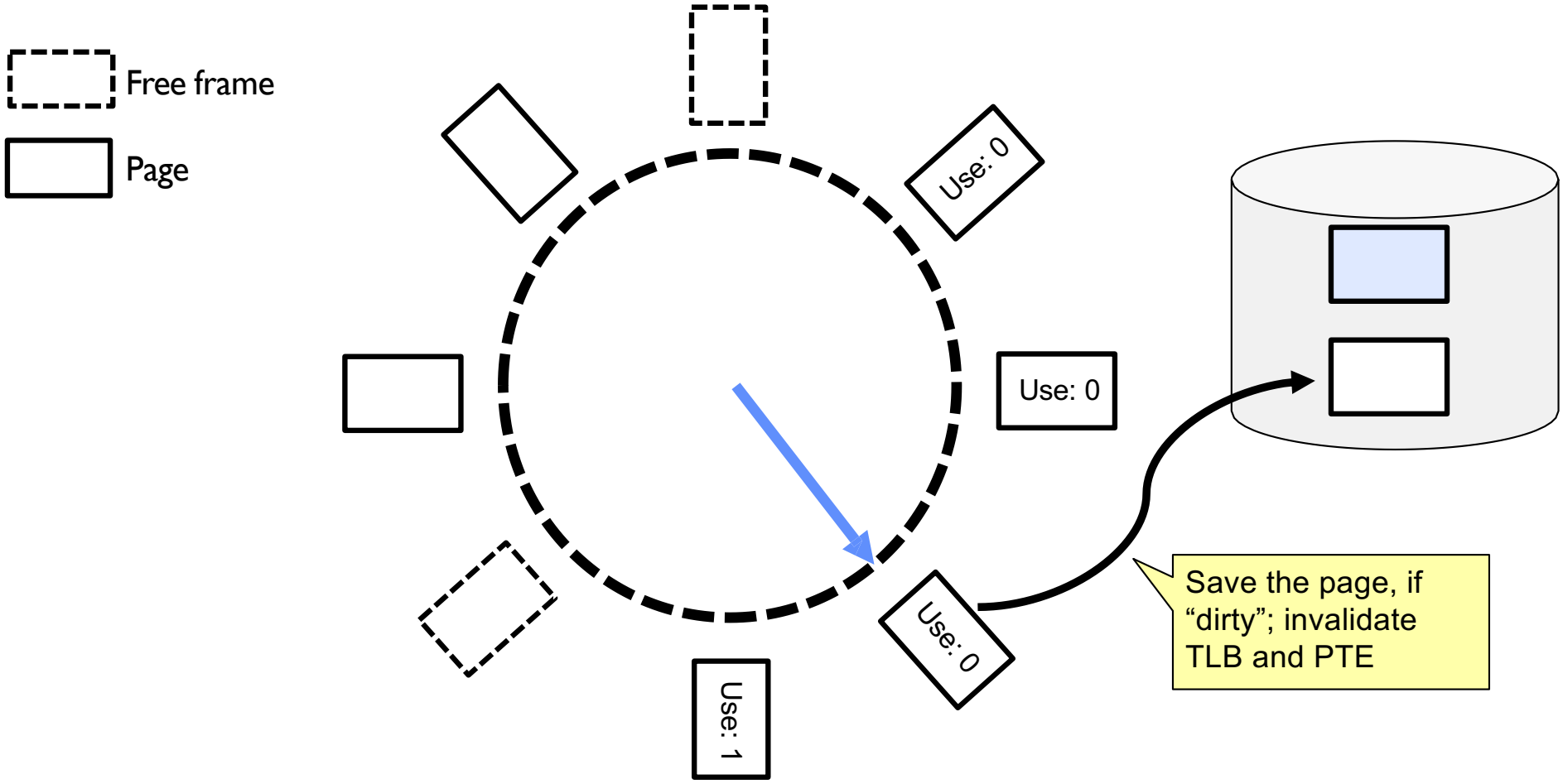
Clock algorithm example: page fault



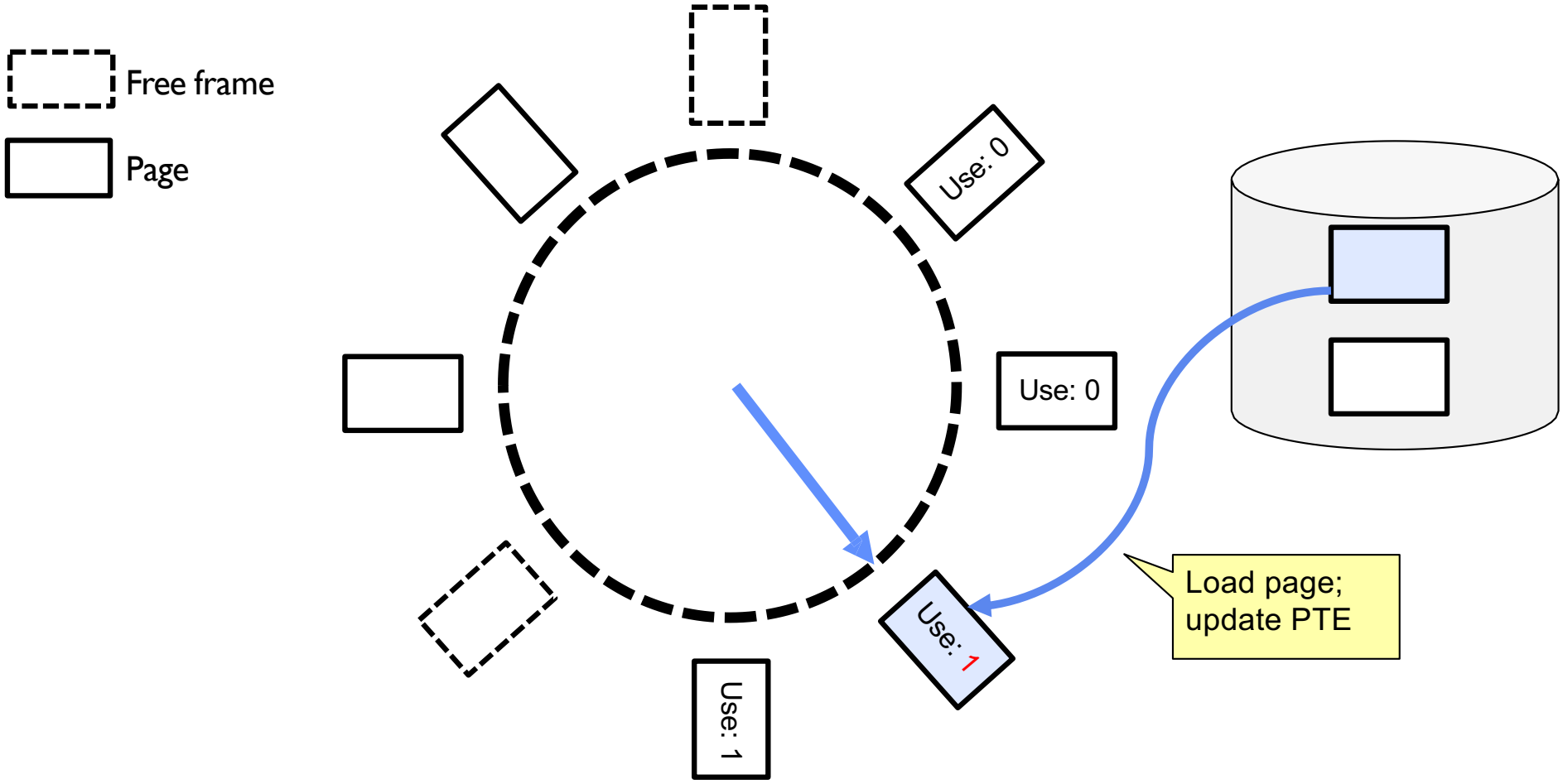
Clock algorithm example: page fault



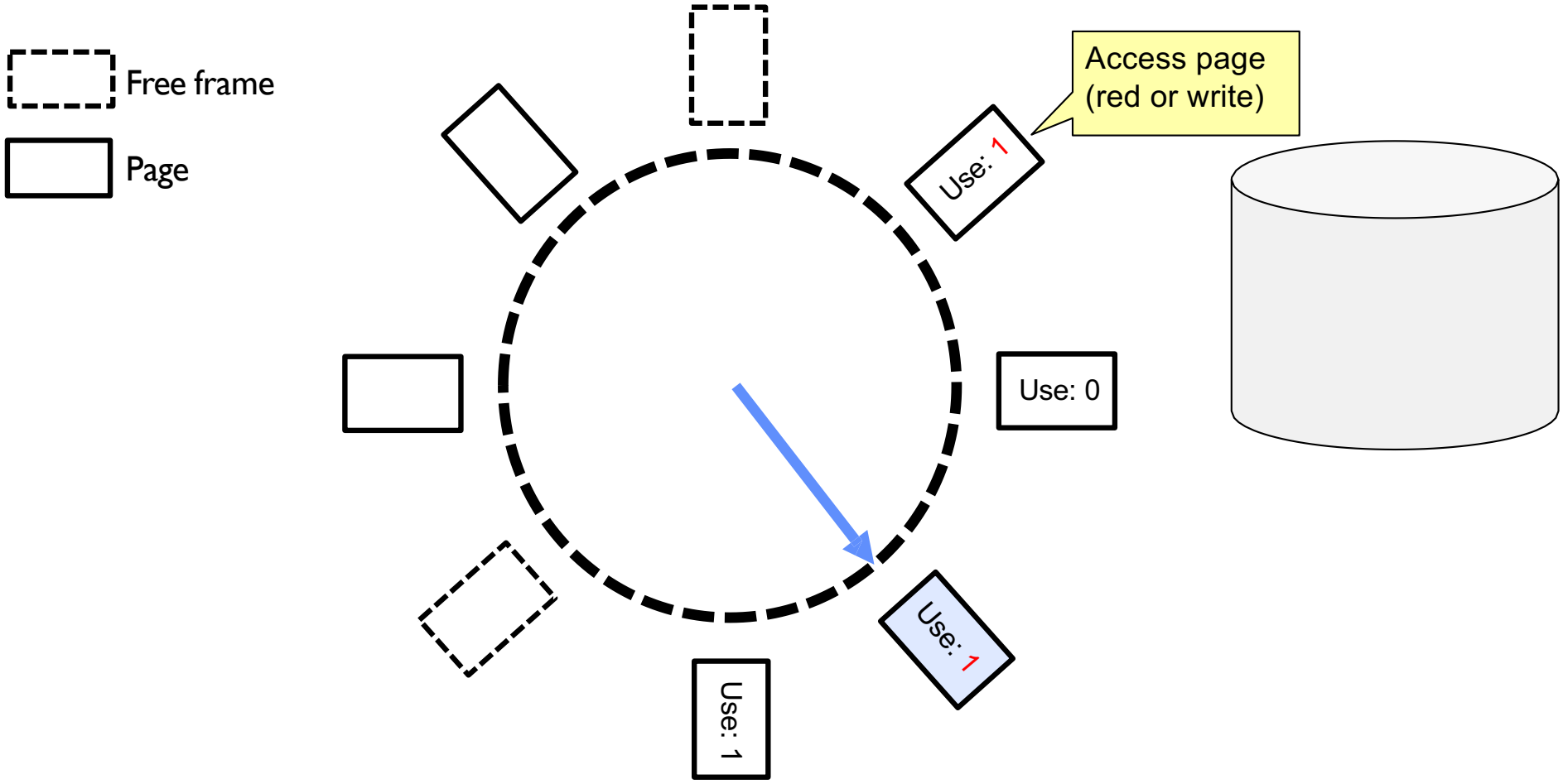
Clock algorithm example: page fault



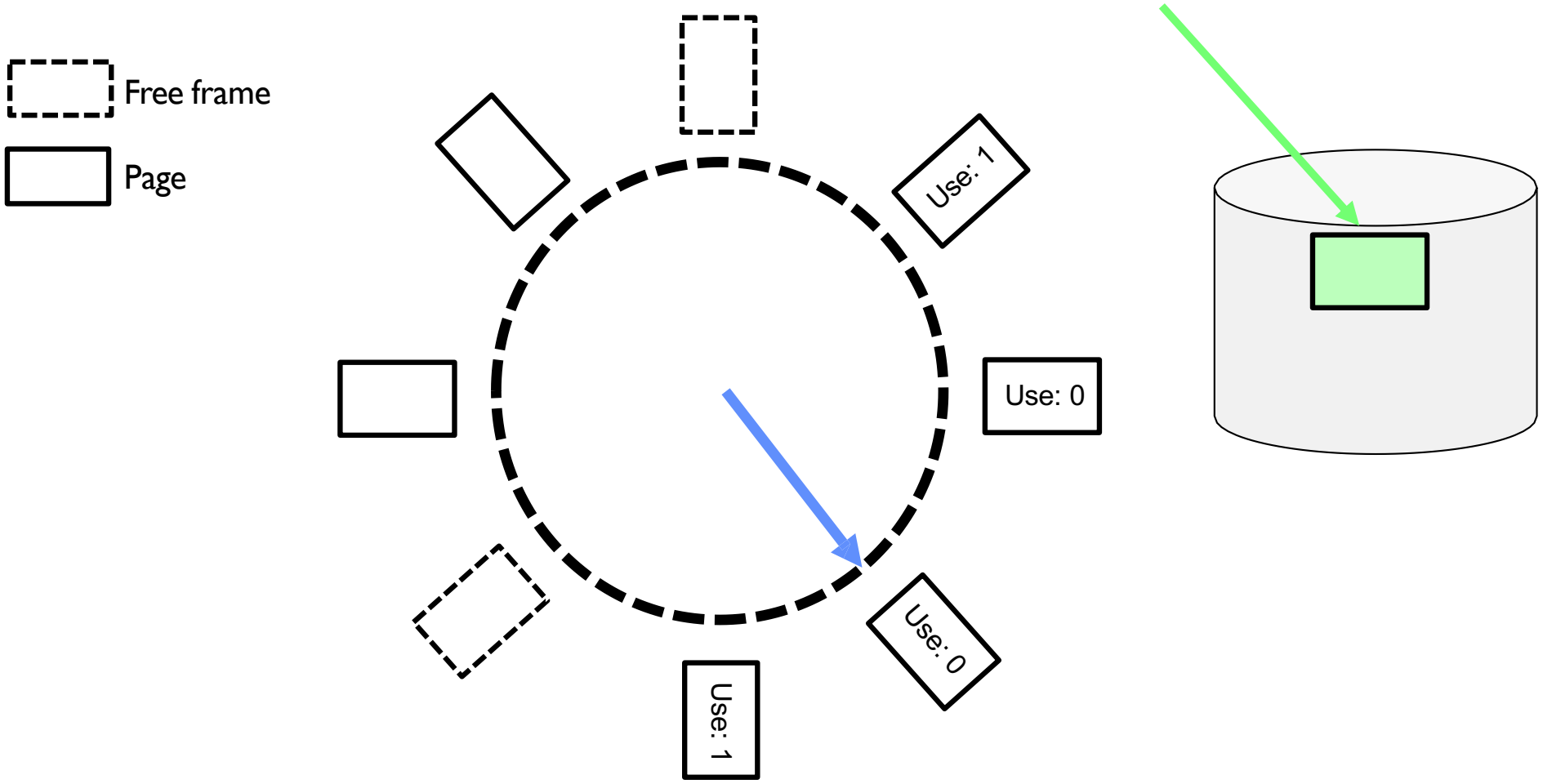
Clock algorithm example: page fault



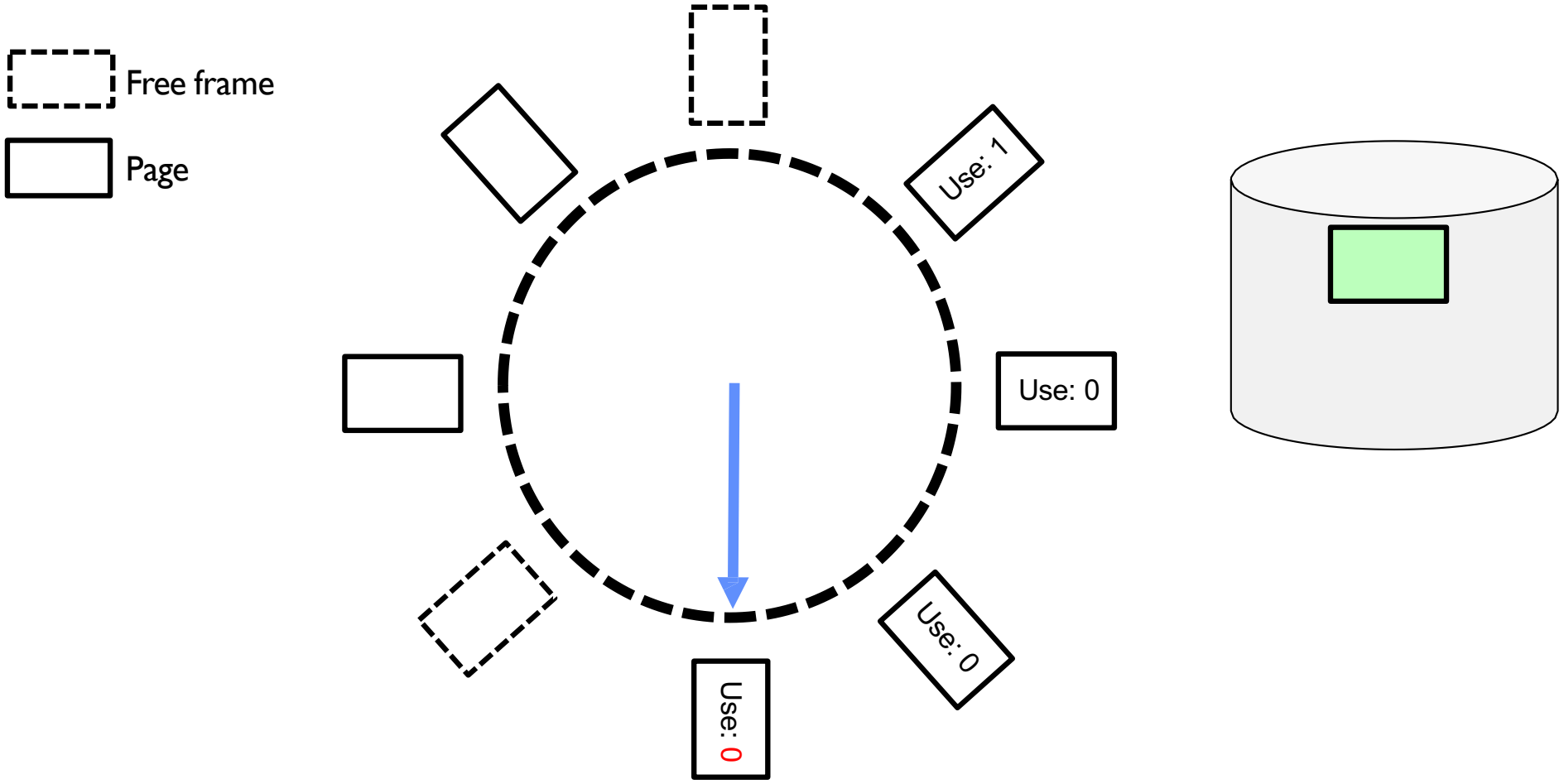
Clock algorithm example



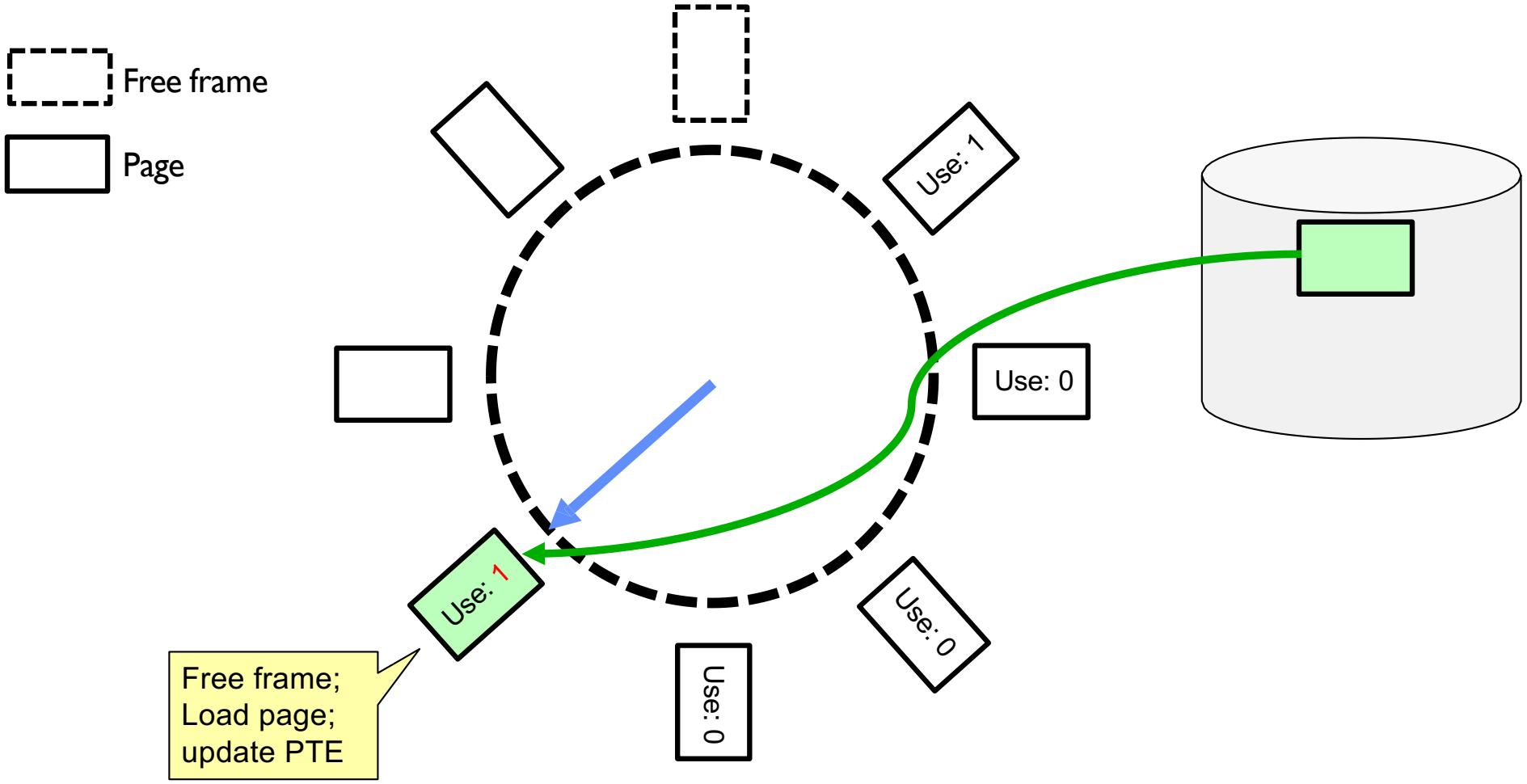
Clock algorithm example: another page fault



Clock algorithm example: another page fault

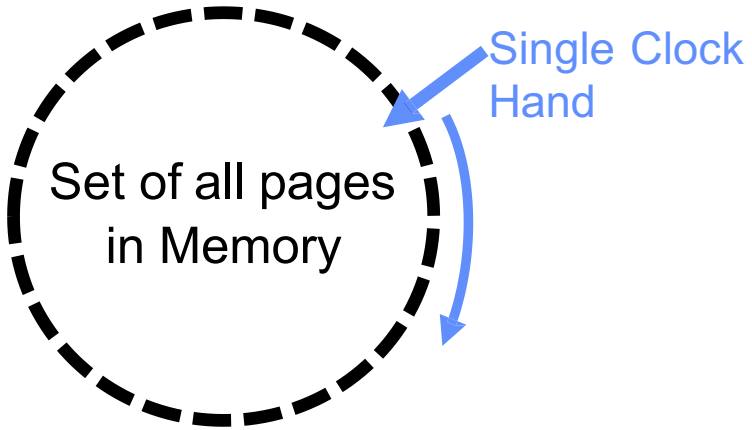


Clock algorithm example: another page fault



Clock algorithm: more details

- Will always find a page or loop forever?
  - Even if all use bits set, will eventually loop all the way around ⇒ FIFO
- What if hand moving slowly?
  - Good sign or bad sign?
    - Not many page faults
    - or find page quickly
- What if hand is moving quickly?
  - Lots of page faults and/or lots of reference bits set
- One way to view clock algorithm:
  - Crude partitioning of pages into two groups: young and old



Nth chance version of clock algorithm

- N^{th} chance algorithm: give page N chances
  - OS keeps counter per page: # sweeps
  - On page fault, OS checks use bit:
    - 1 → clear use and also clear counter (used in last sweep)
    - 0 → increment counter; if count=N, replace page
  - Means that clock hand might have to sweep by N times without page being used before page is replaced
- How do we pick N?
  - Why pick large N? Better approximation to LRU
    - If N ~ 1K, really good approximation
  - Why pick small N? More efficient
    - Otherwise, might have to look a long way to find free page

Nth chance version of clock algorithm

- What about “modified” (or “dirty”) pages?
  - Takes extra overhead to replace a dirty page, so give dirty pages an extra chance before replacing?
  - Common approach:
    - Clean pages, use N=1
    - Dirty pages, use N=2 (and write back to disk when N=1)

Clock algorithms variations

- Do we really need hardware-supported “modified” (“dirty”) bit?
  - No. We can emulate it using read-only bit
    - Need software “database” of which pages are allowed to be written (needed this anyway)
    - We will tell MMU that pages have more restricted permissions than the actually do to force page faults (and allow us notice when page is written)
  - Algorithm (Clock-Emulated-M):
    - Initially, mark all pages as read-only (W→0), even writable data pages. Further, clear all software versions of the “modified” bit → 0 (page not dirty)
    - Writes will cause a page fault. Assuming write is allowed, OS sets software “modified” bit → 1, and marks page as writable (W→1).
    - Whenever page written back to disk, clear “modified” bit → 0, mark read-only

Clock algorithms variations (continued)

- Do we really need a hardware-supported “use” bit?
  - No. Can emulate it similar to above (e.g. for read operation)
    - Kernel keeps a “use” bit and “modified” bit for each page

Clock algorithms variations (continued)

- Algorithm (Clock-Emulated-Use-and-M):
  - Mark all pages as invalid, even if in memory. Clear emulated "use" bits → 0 and "modified" bits → 0 for all pages (not used, not dirty)
  - Read or write to invalid page traps to OS to tell use page has been used
  - OS sets "use" bit → 1 in software to indicate that page has been "used". Further:
    1. If read, mark page as read-only, W→0 (will catch future writes)
    2. If write (and write allowed), set "modified" bit → 1, mark page as writable (W→1)
  - When clock hand passes, reset emulated "use" bit → 0 and mark page as invalid again
  - Note that "modified" bit left alone until page written back to disk

Clock algorithms variations (continued)

- Remember, however, clock is just an approximation of LRU!
  - Can we do a better approximation, given that we have to take page faults on some reads and writes to collect use information?
  - Need to identify an old page, not oldest page!

Conclusion

- Replacement policies
  - FIFO: Place pages on queue, replace page at end
  - MIN: Replace page that will be used farthest in future
  - LRU: Replace page used farthest in past
- Working set:
  - Set of pages touched by a process recently
  - Point of Replacement algorithms is to try to keep working set in memory
- Clock algorithm: approximation to LRU
  - Arrange all pages in circular list
  - Sweep through them, marking as not “in use”
  - If page not “in use” for one pass, than can replace
- N^{th}-chance clock algorithm: another approximate LRU
  - Give pages multiple passes of clock hand before replacing

Modern computer systems: memory

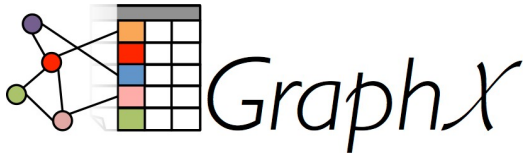
Efficient Memory Disaggregation with Infiniswap

Juncheng Gu, Youngmoon Lee, Yiwen Zhang,
Mosharaf Chowdhury, Kang G. Shin

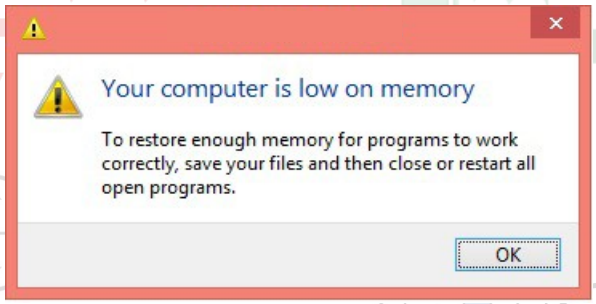
Agenda

- **Motivation and related work**
- Design and system overview
- Implementation and evaluation
- Future work and conclusion

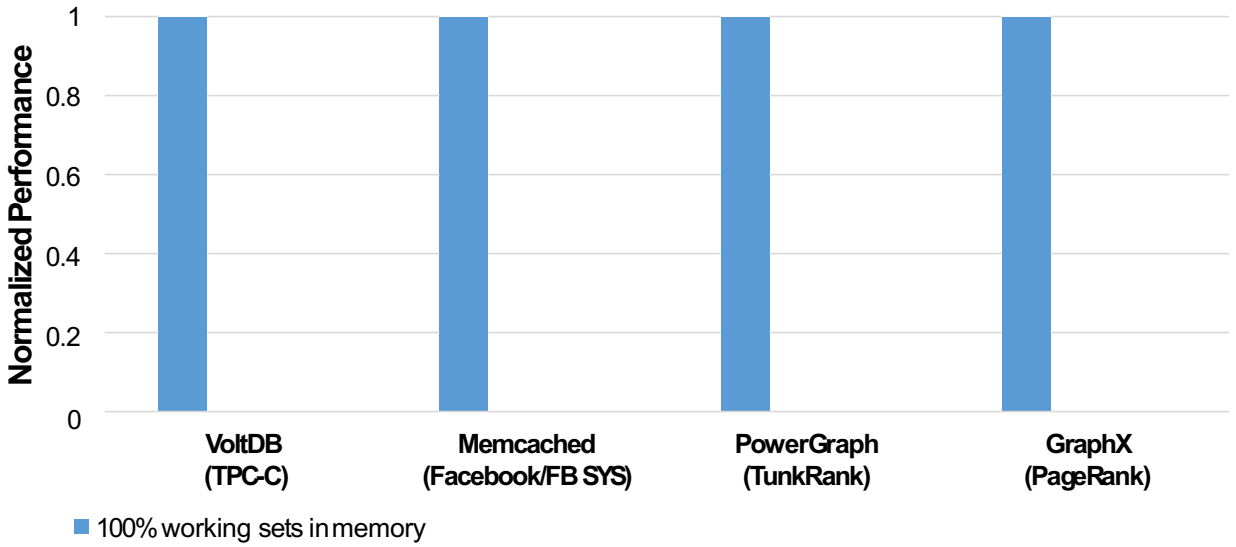
Memory-intensive applications



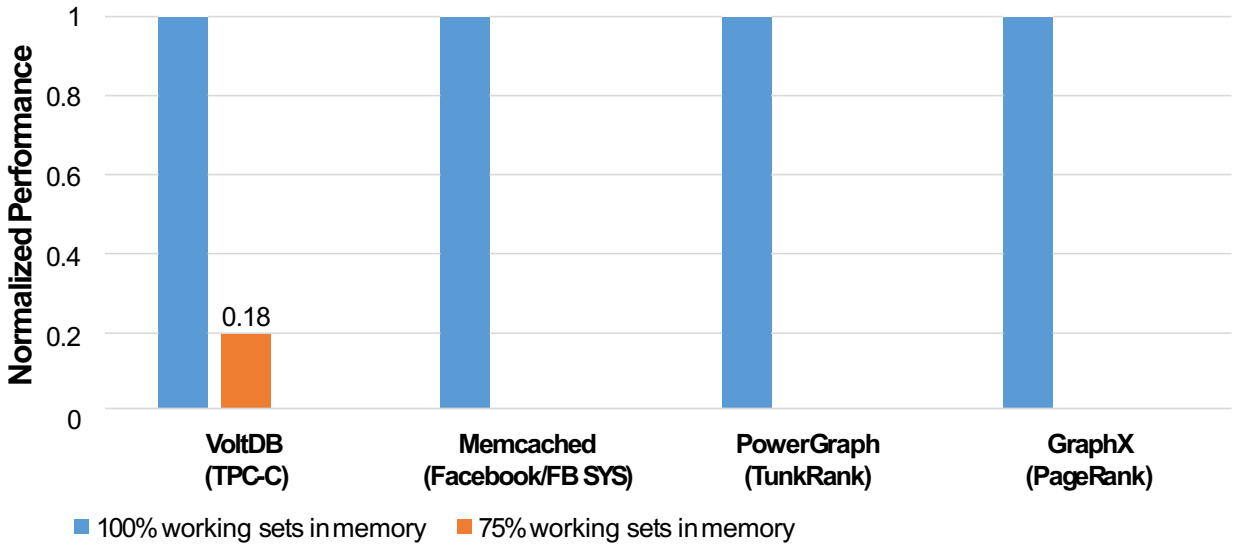
Memory-intensive applications



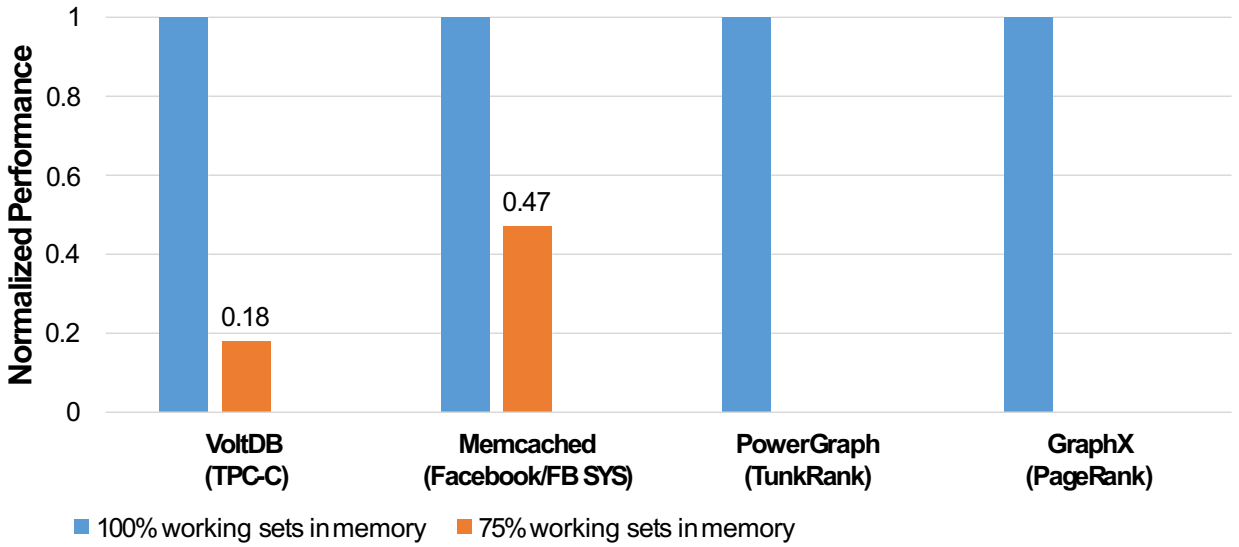
Performance degradation



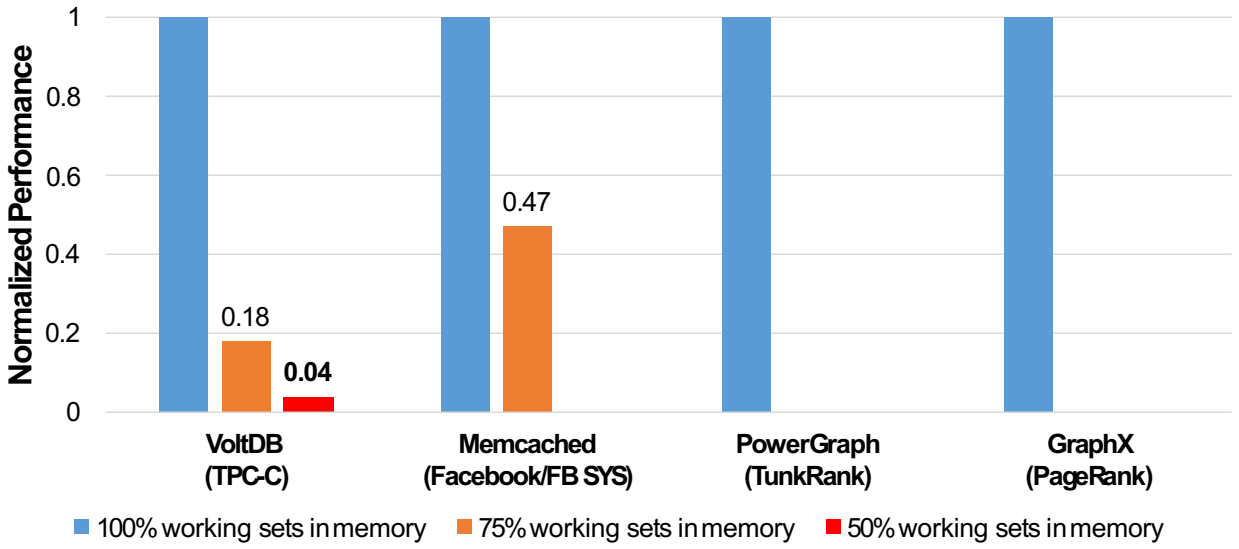
Performance degradation



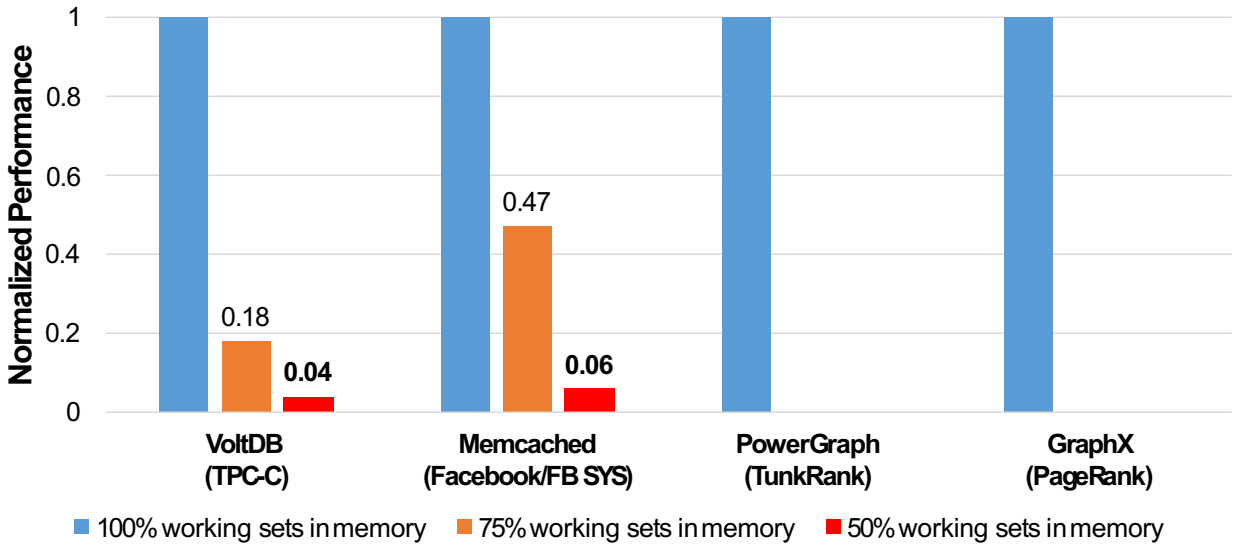
Performance degradation



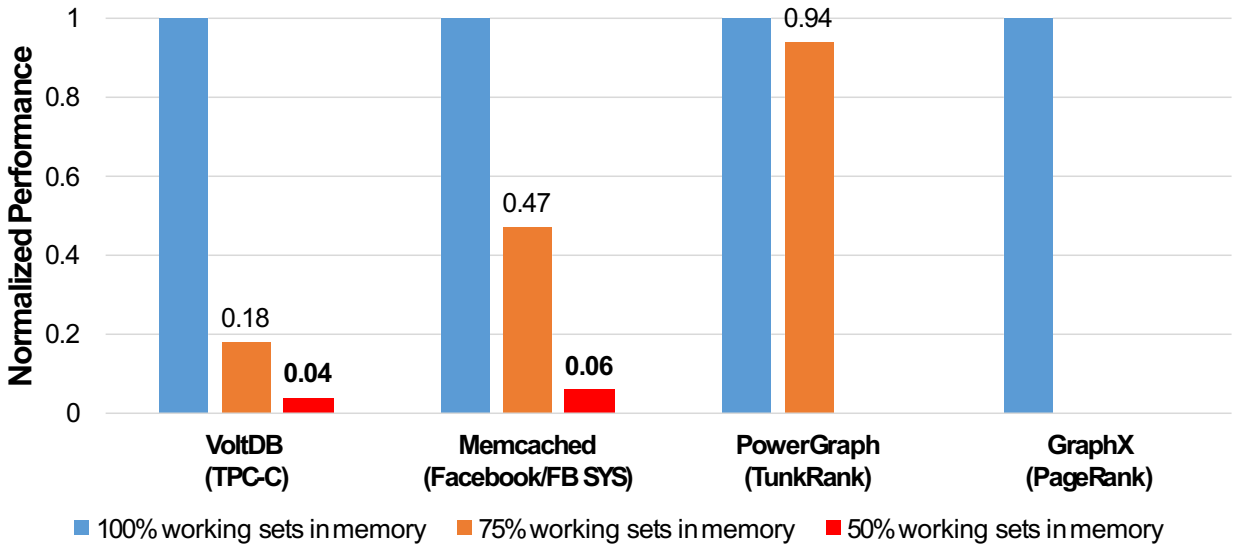
Performance degradation



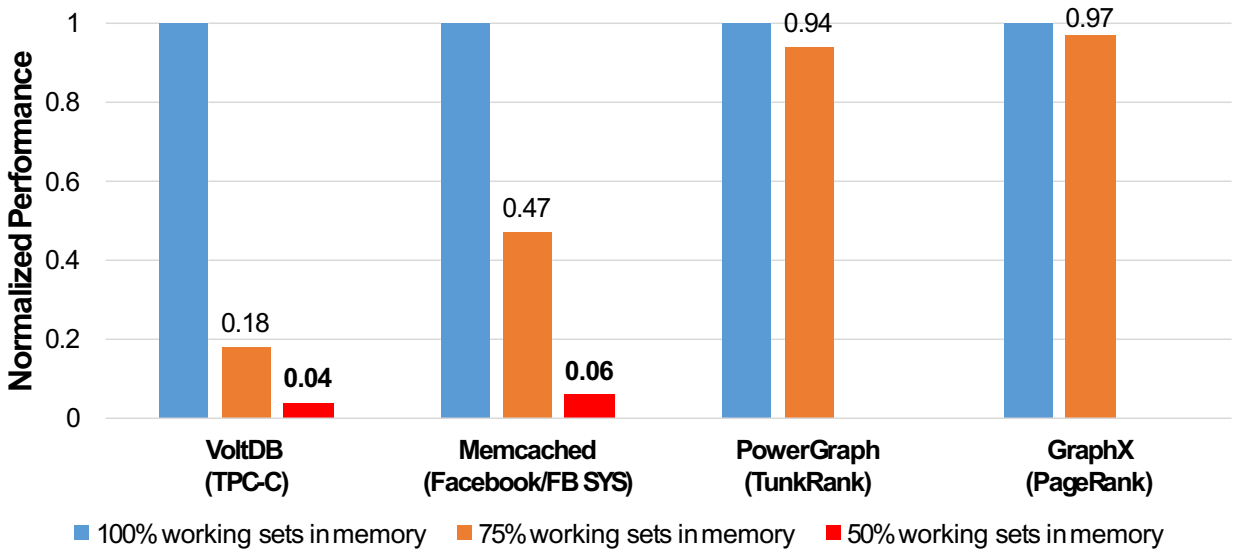
Performance degradation



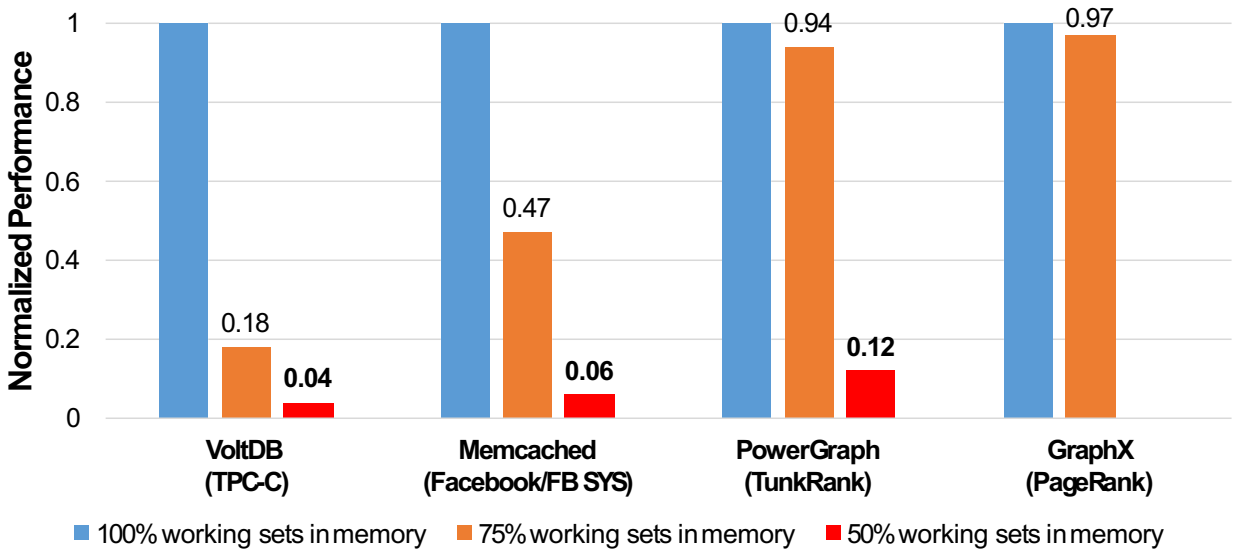
Performance degradation



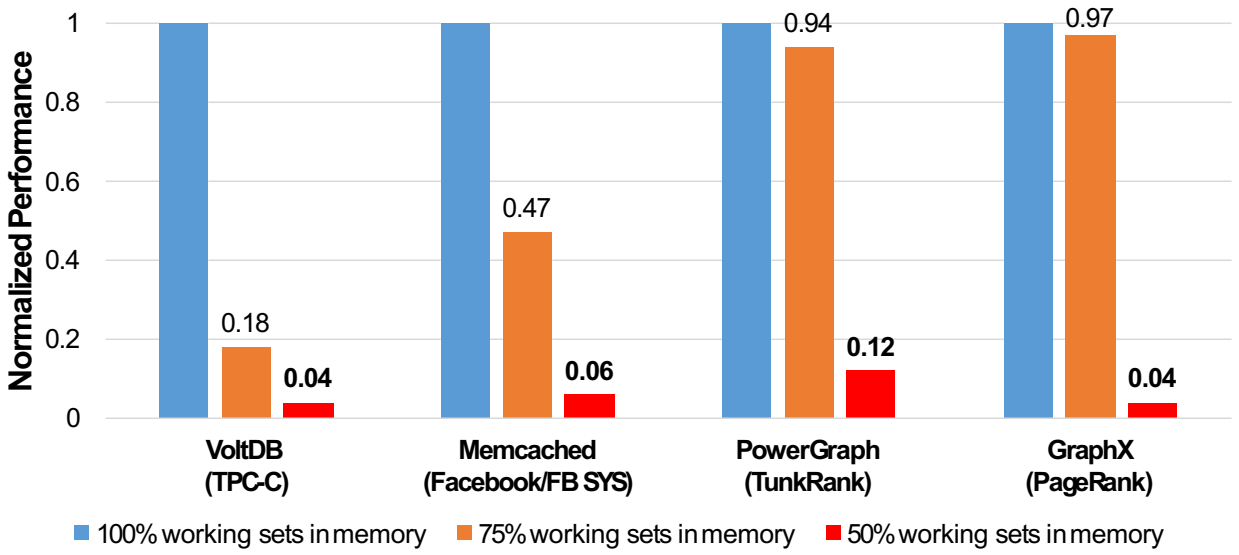
Performance degradation



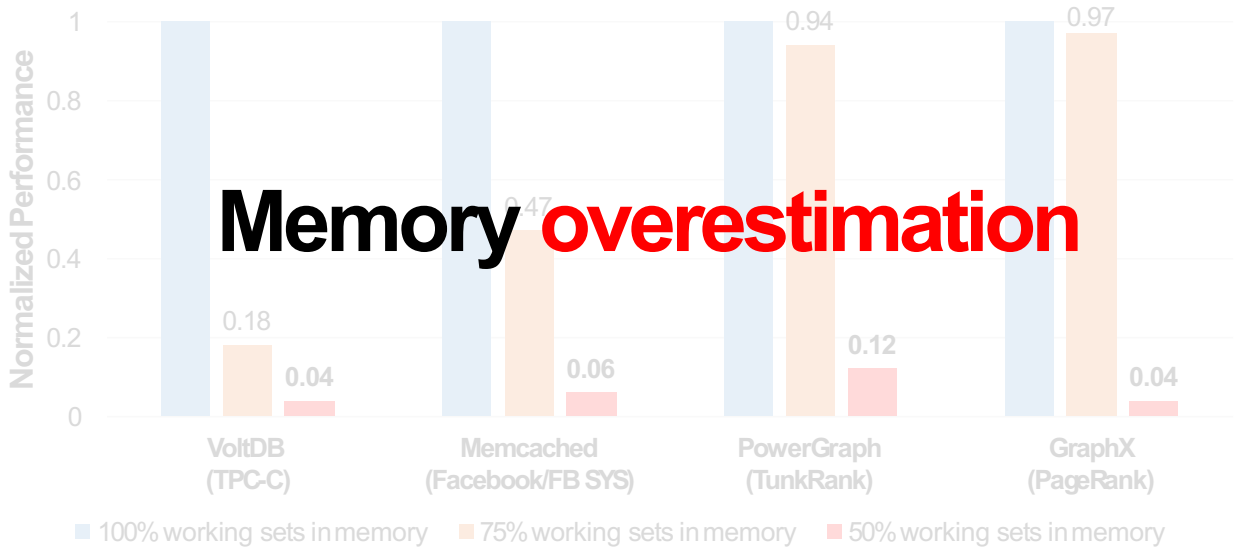
Performance degradation



Performance degradation



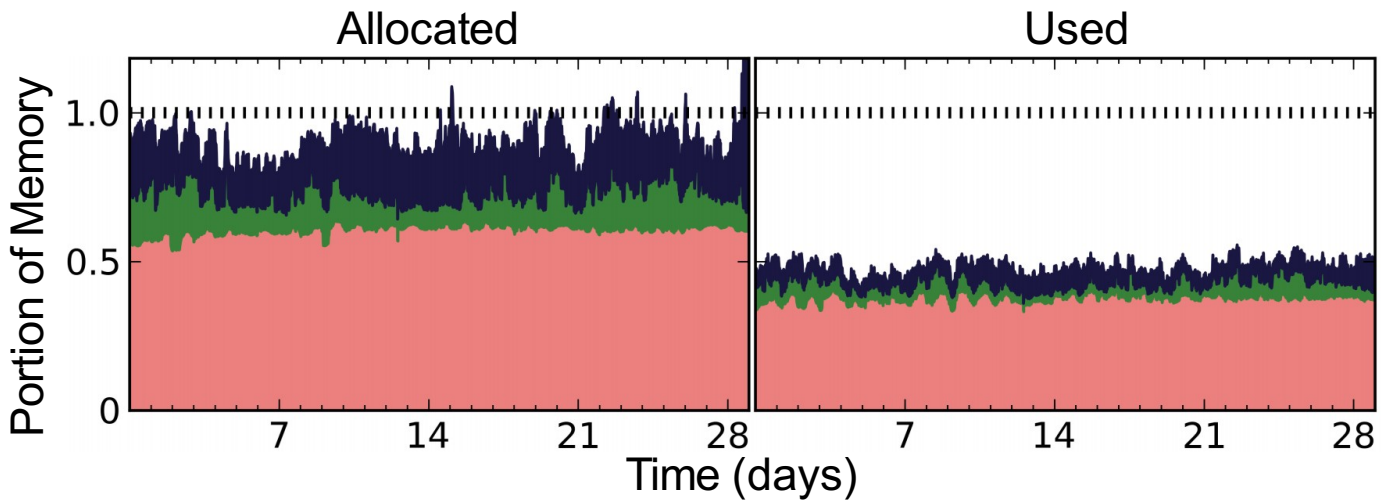
Performance degradation



Memory overestimation

Memory underutilization

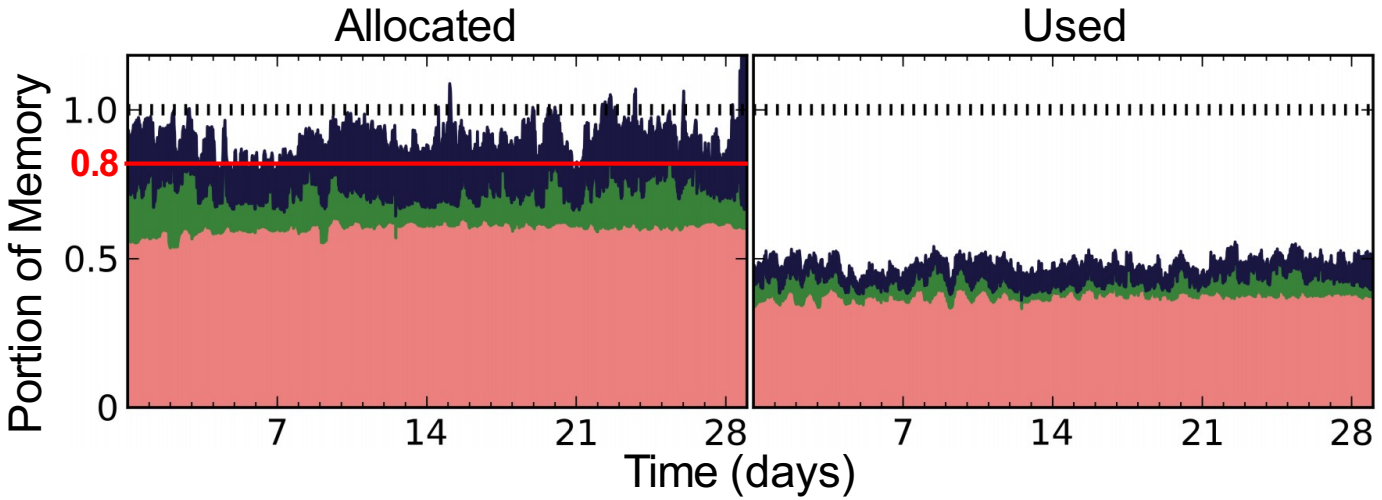
- Google Cluster Analysis[1]



[1] Reiss, Charles, et al. "Heterogeneity and dynamicity of clouds at scale: Google trace analysis." *SoCC'12*.

Memory underutilization

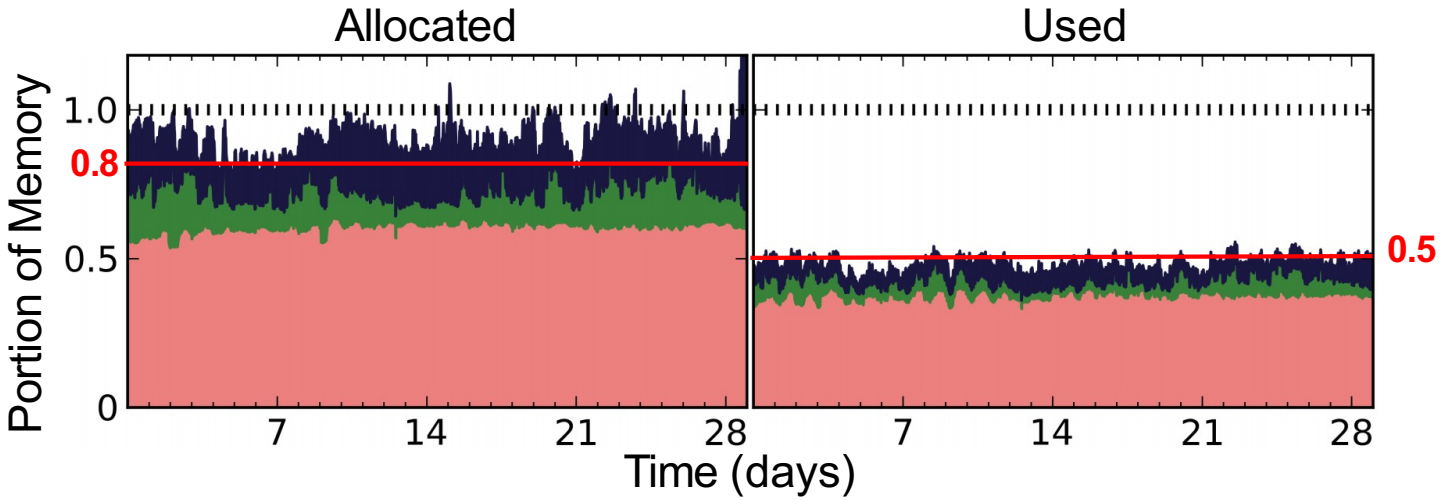
- Google Cluster Analysis[1]



[1] Reiss, Charles, et al. "Heterogeneity and dynamicity of clouds at scale: Google trace analysis." *SoCC'12*.

Memory underutilization

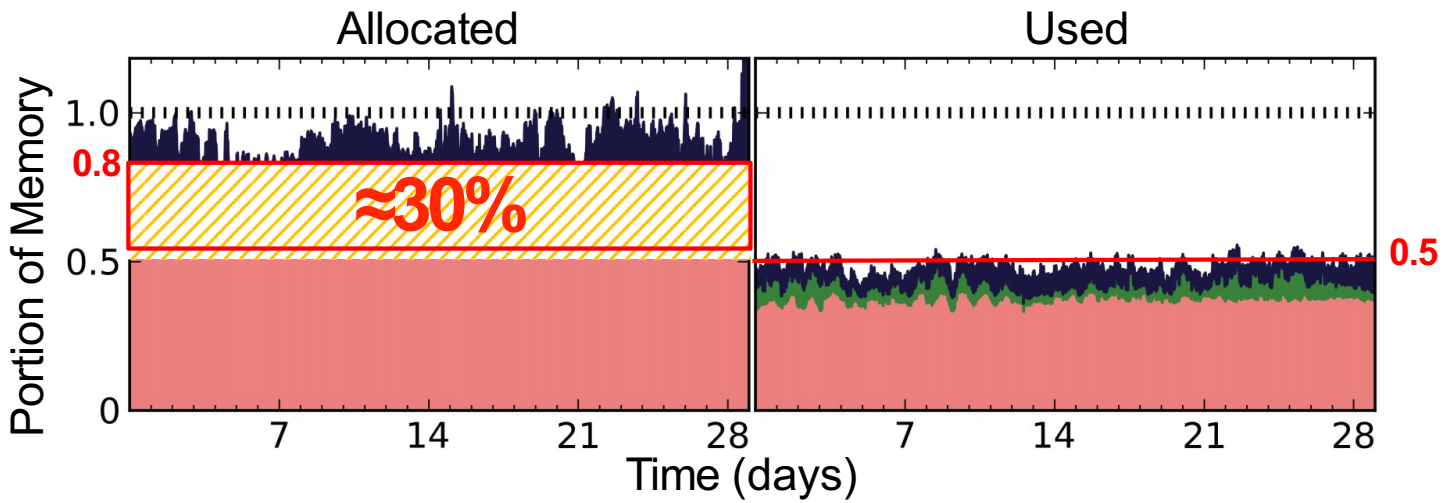
- Google Cluster Analysis[1]



[1] Reiss, Charles, et al. "Heterogeneity and dynamicity of clouds at scale: Google trace analysis." *SoCC'12*.

Memory underutilization

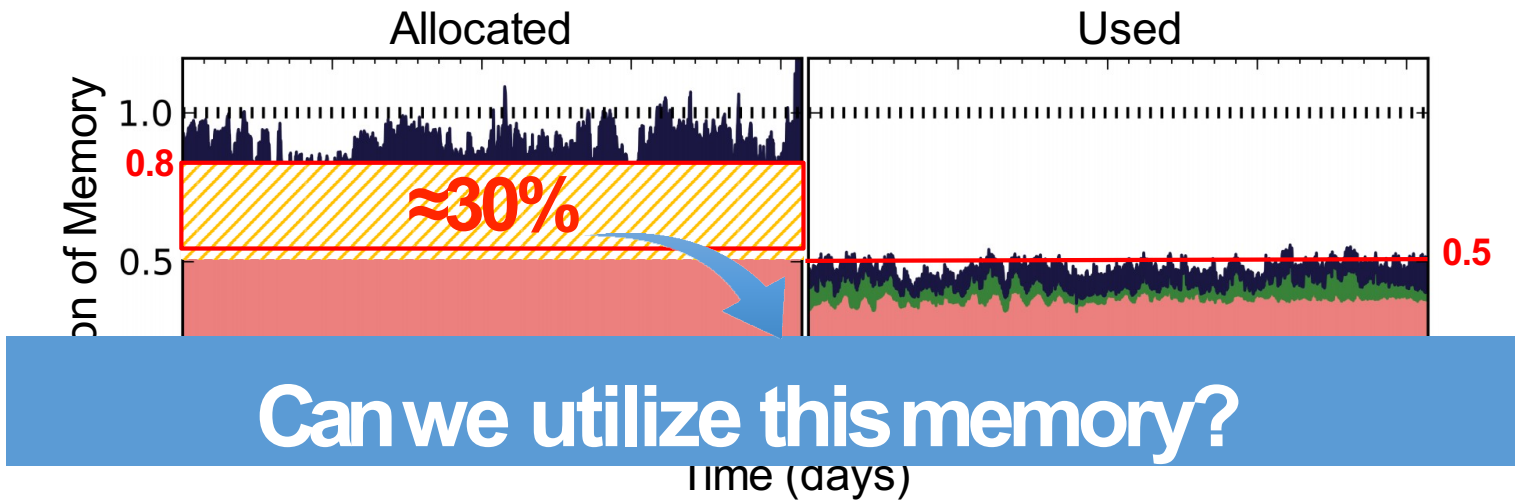
- Google Cluster Analysis[1]



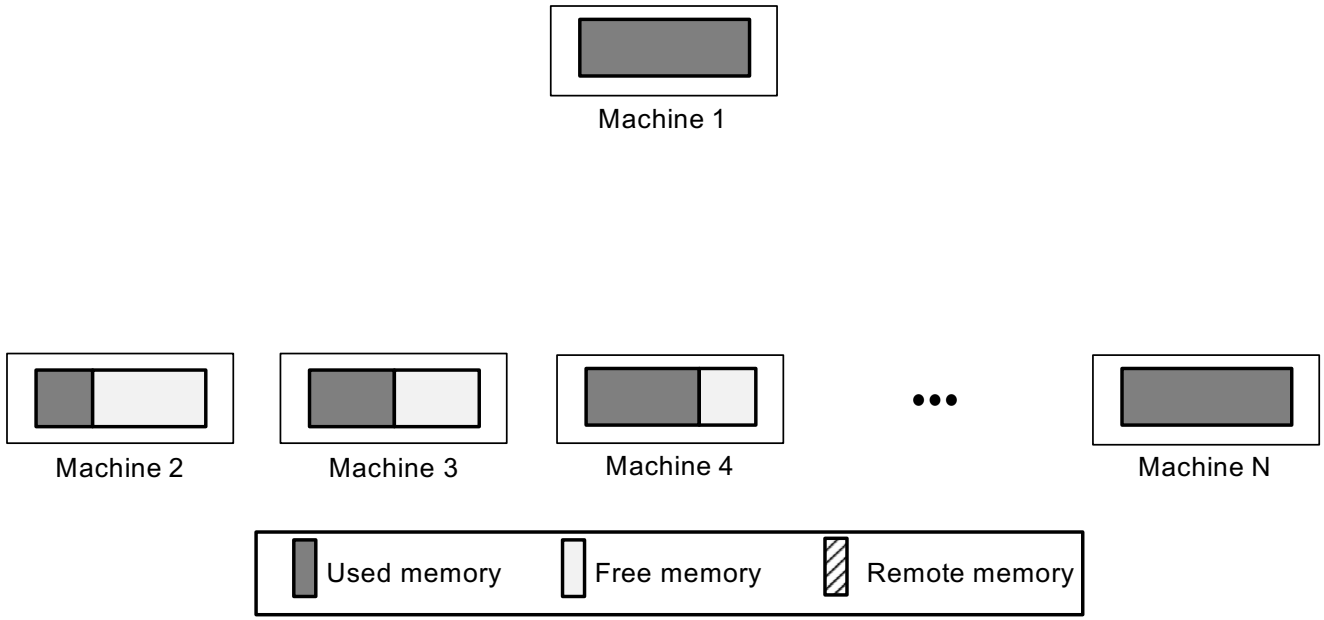
[1] Reiss, Charles, et al. "Heterogeneity and dynamicity of clouds at scale: Google trace analysis." *SoCC'12*.

Memory underutilization

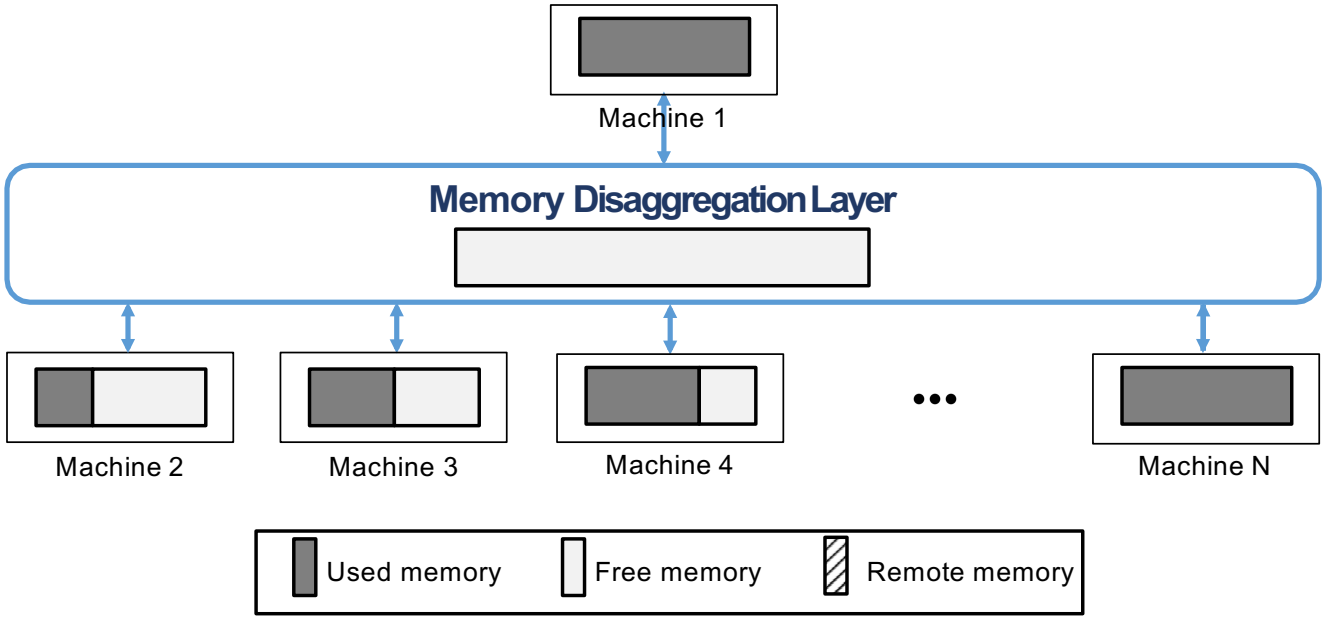
- Google Cluster Analysis[1]



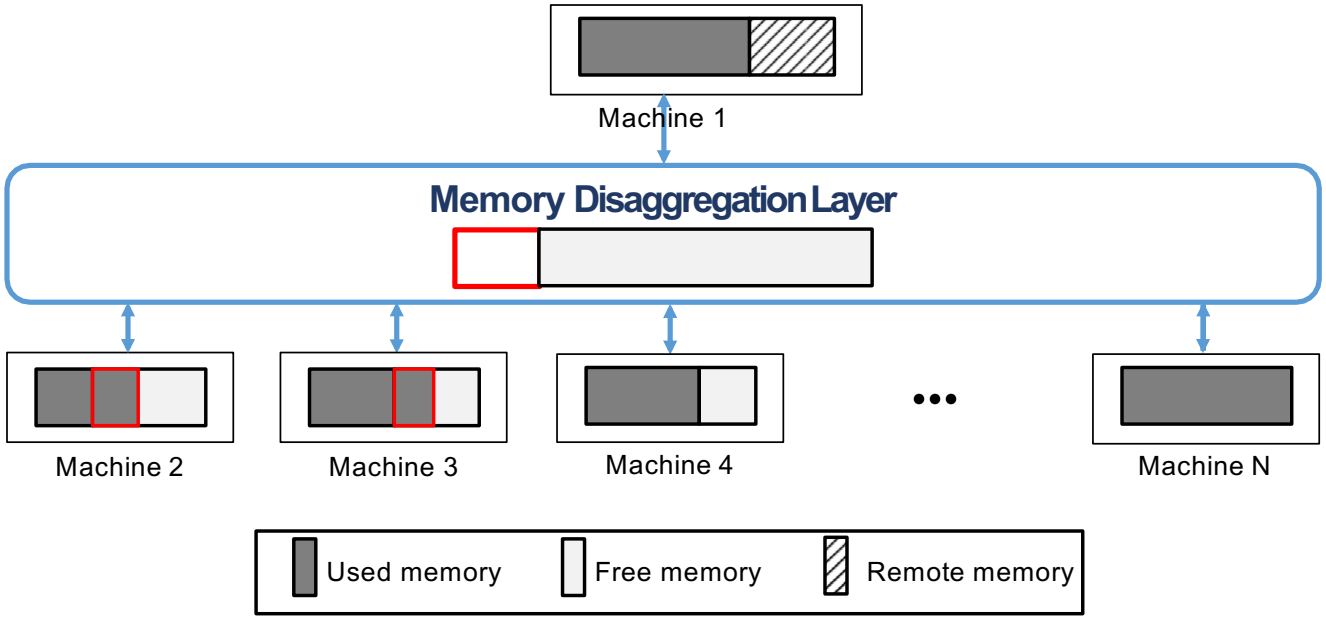
[1] Reiss, Charles, et al. "Heterogeneity and dynamicity of clouds at scale: Google trace analysis." *SoCC'12*.


Machine 1
Machine 2
Machine 3
Machine 4
Machine N
Used memory
Free memory
Remote memory

Disaggregate free memory



Disaggregate free memory



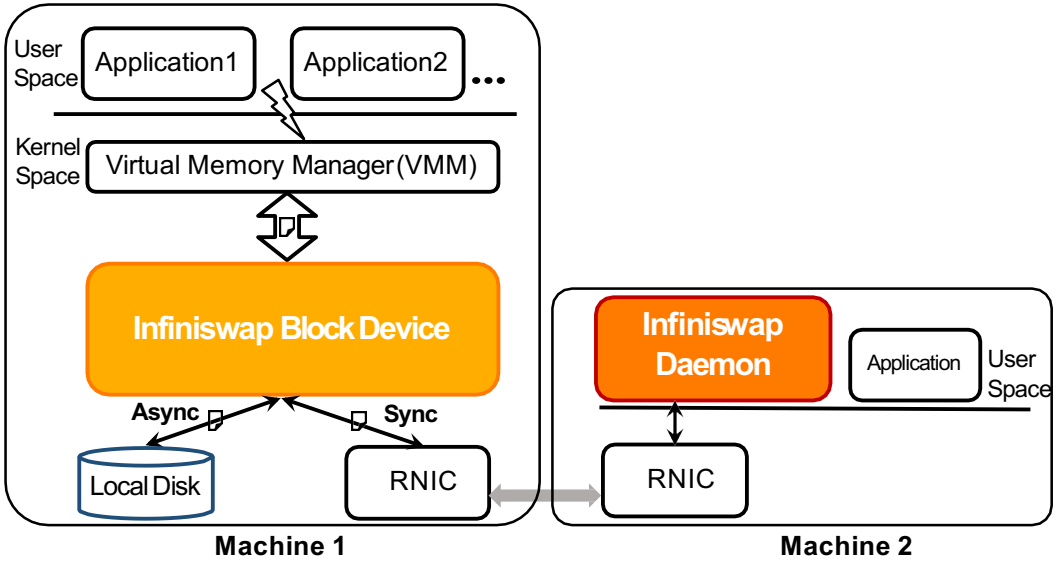
What are the challenges?

- **Minimize deployment overhead**
  - **No hardware design**
  - **No application modification**
- **Tolerate failures**
  - e.g. network disconnection, machine crash
- **Manage remote memory at scale**

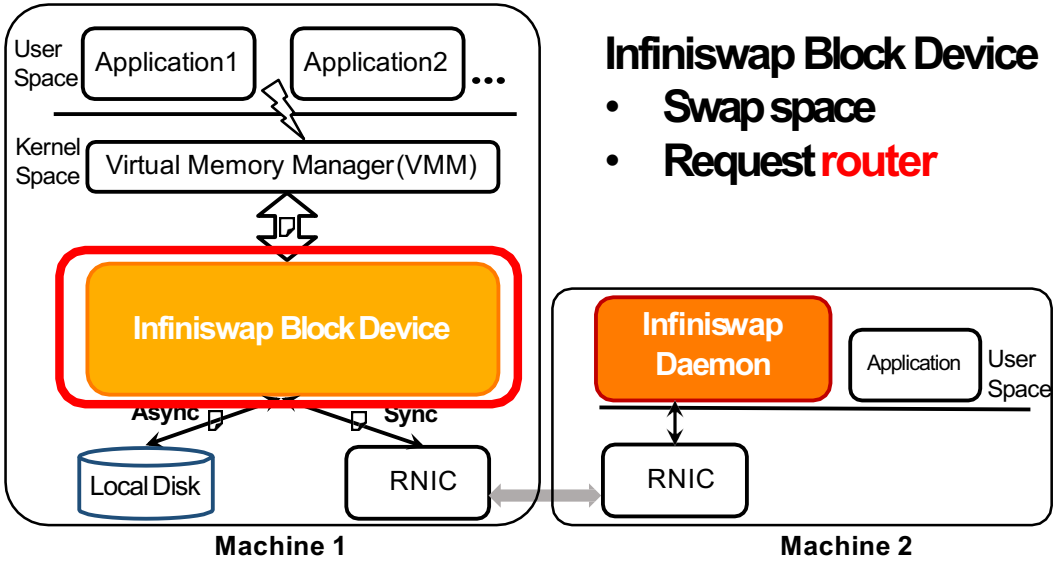
Agenda

- Motivation and related work
- **Design and system overview**
- Implementation and evaluation
- Future work and conclusion

System Overview



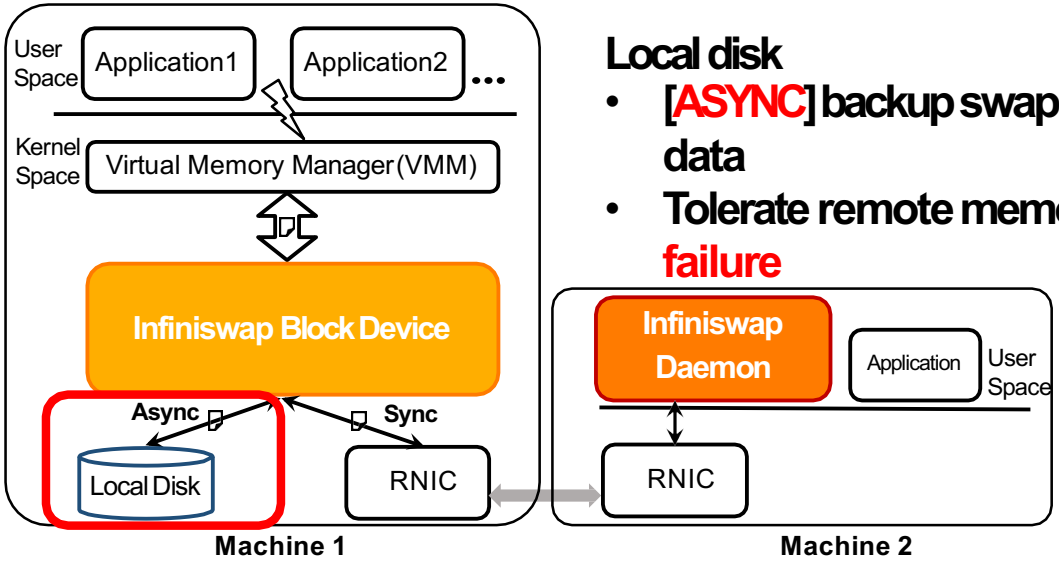
System Overview



Infiniswap Block Device

- **Swap space**
- **Request router**

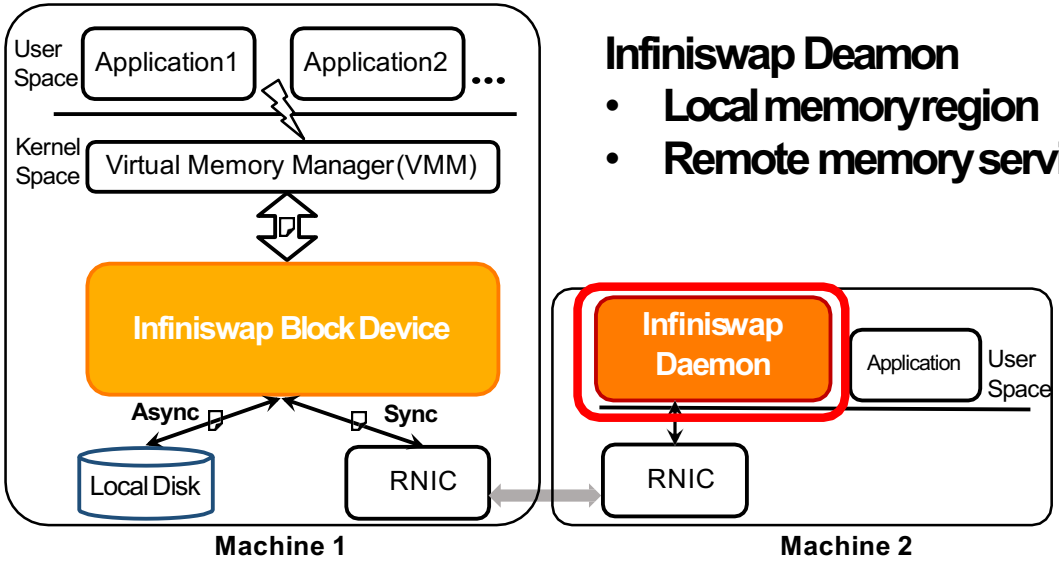
System Overview



Local disk

- **[ASYNC] backup swapped-out data**
- **Tolerate remote memory failure**

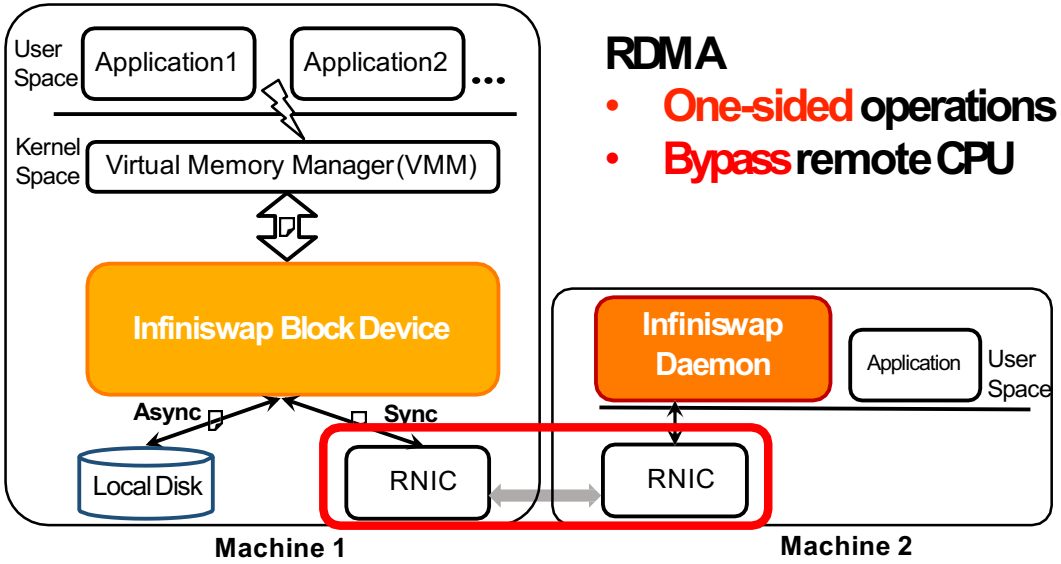
System Overview



Infiniswap Deamon

- **Local memory region**
- **Remote memory service**

System Overview



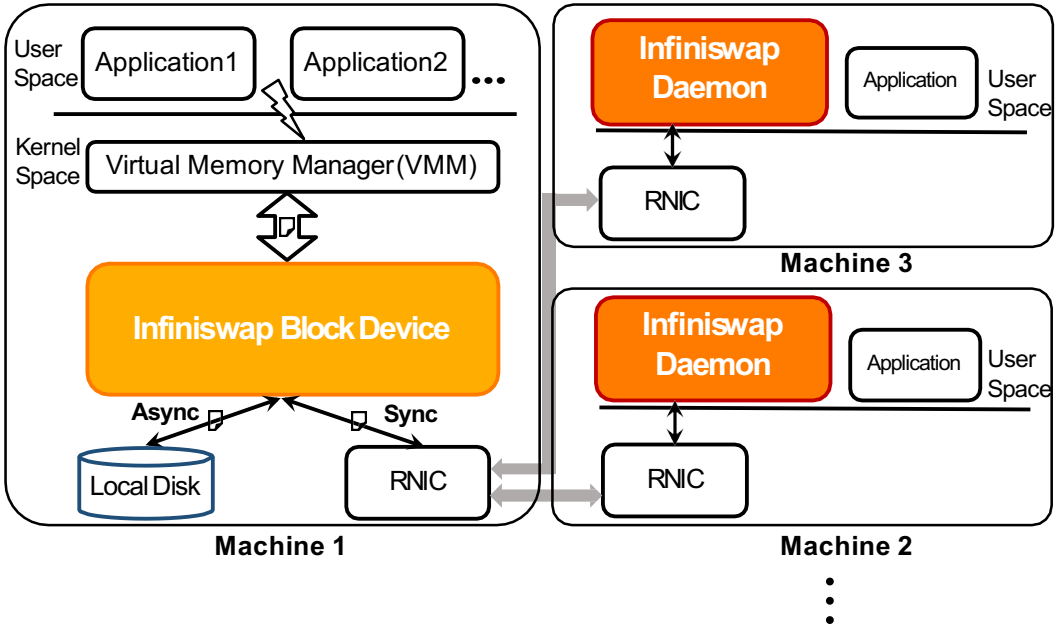
RDMA

- **One-sided operations**
- **Bypass remote CPU**

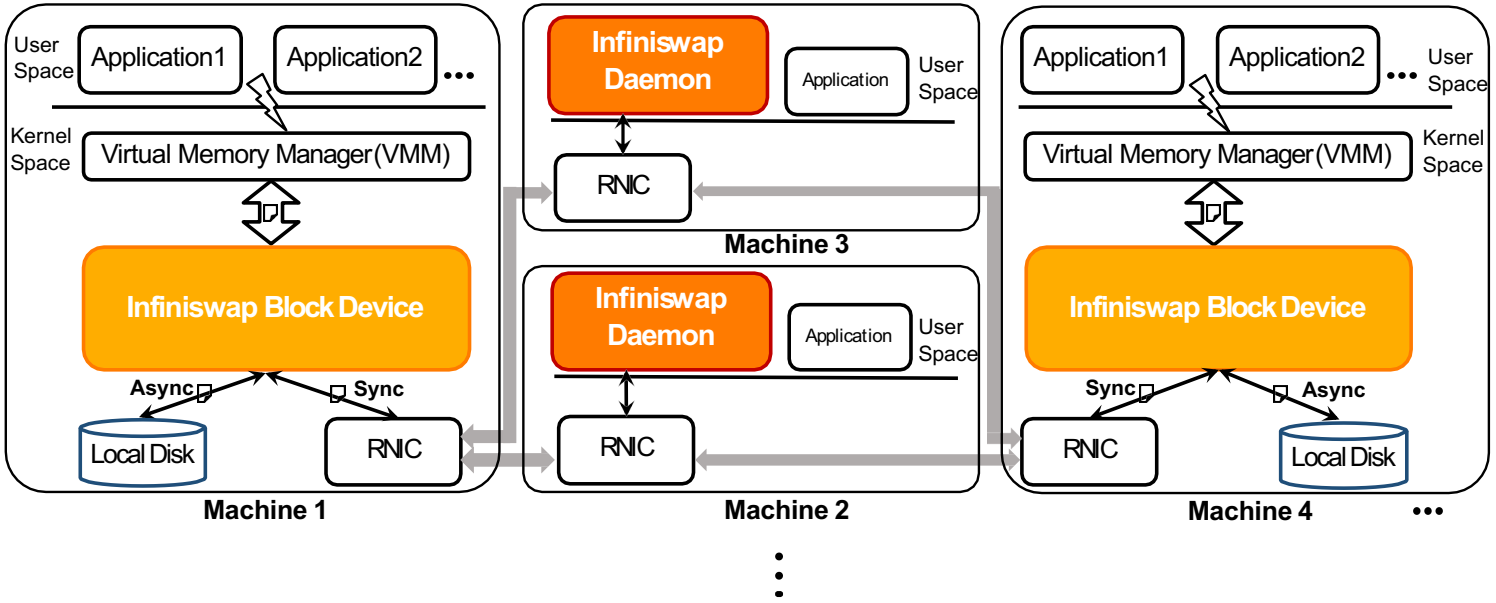
How to meet the design objectives?

Objectives	Ideas
No hardware design	Remote paging
No application modification	
Fault-tolerance	Local backup disk

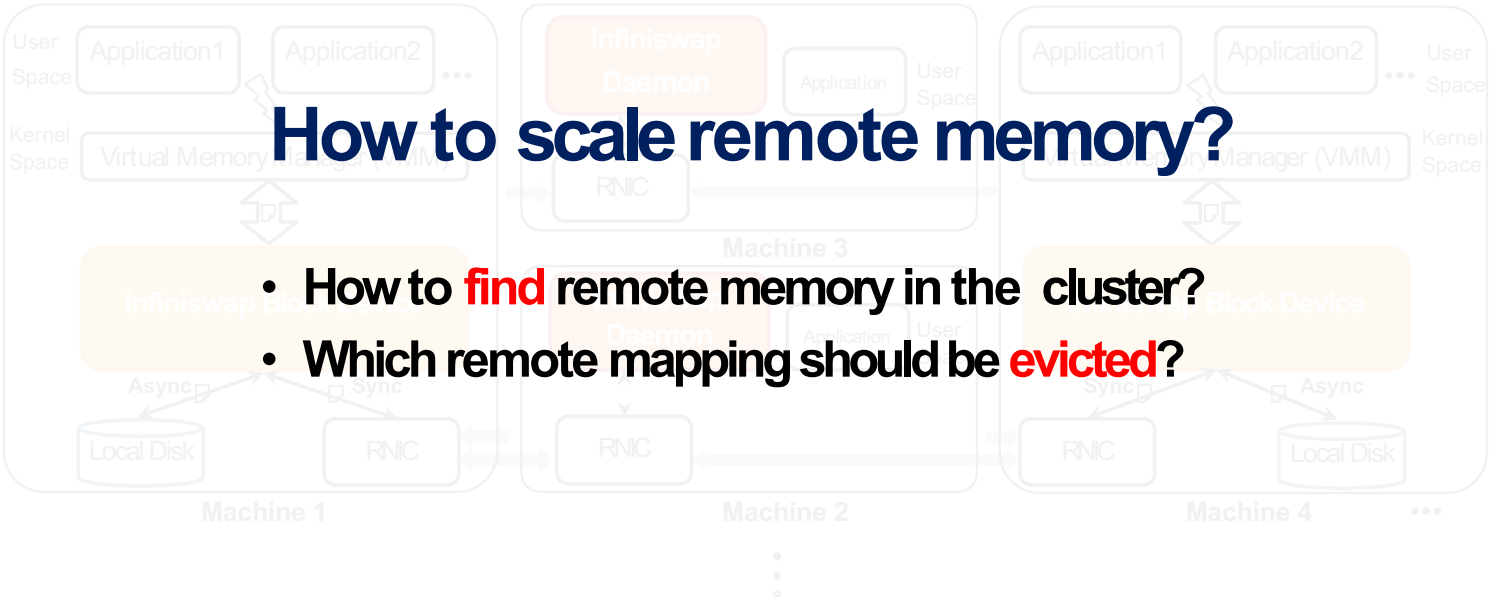
One-to-many


User Space
Application1
Application2
Kernel Space
Virtual Memory Manager (VMM)
Infiniswap Block Device
Async
Sync
Local Disk
RNIC
Machine 1
Infiniswap Daemon
Application
User Space
RNIC
Machine 3
Infiniswap Daemon
Application
User Space
RNIC
Machine 2

Many-to-many


User Space
Application1
Application2
Kernel Space
Virtual Memory Manager(VMM)
Infiniswap Block Device
Async
Sync
Local Disk
RNIC
Machine 1
Infiniswap Daemon
Application
User Space
RNIC
Machine 3
Infiniswap Daemon
Application
User Space
RNIC
Machine 2
Application1
Application2
User Space
Kernel Space
Virtual Memory Manager(VMM)
Infiniswap Block Device
Sync
Async
RNIC
Local Disk
Machine 4

Many-to-many



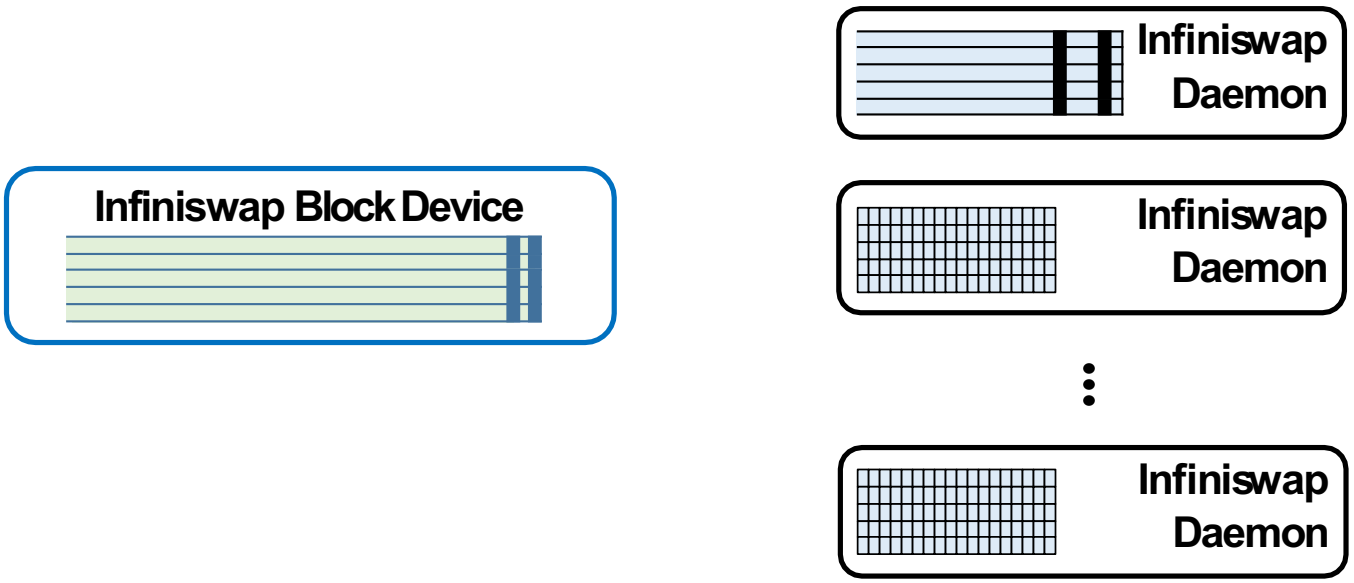
How to scale remote memory?

- How to **find** remote memory in the cluster?
- Which remote mapping should be **evicted**?

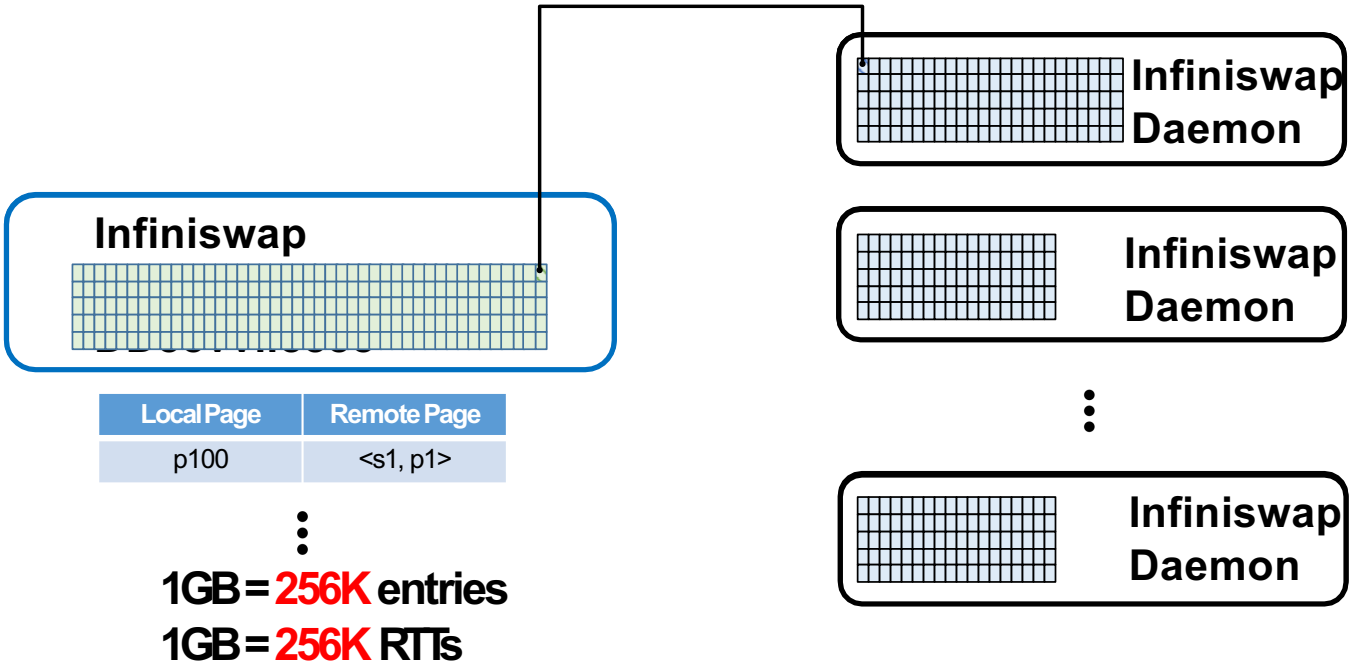
How to meet the design objectives?

Objectives	Ideas
No hardware design	Remote paging
No application modification	
Fault-tolerance	Local backup disk
Scalability	**Decentralized** remote memory management

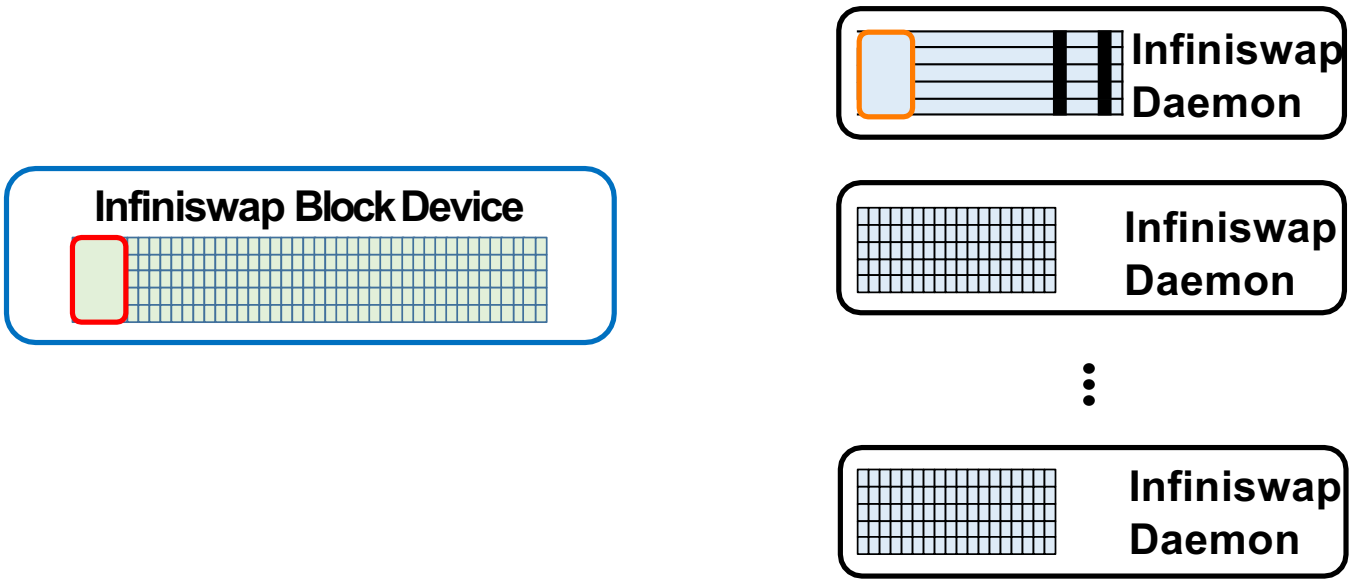
Management unit: memory page?



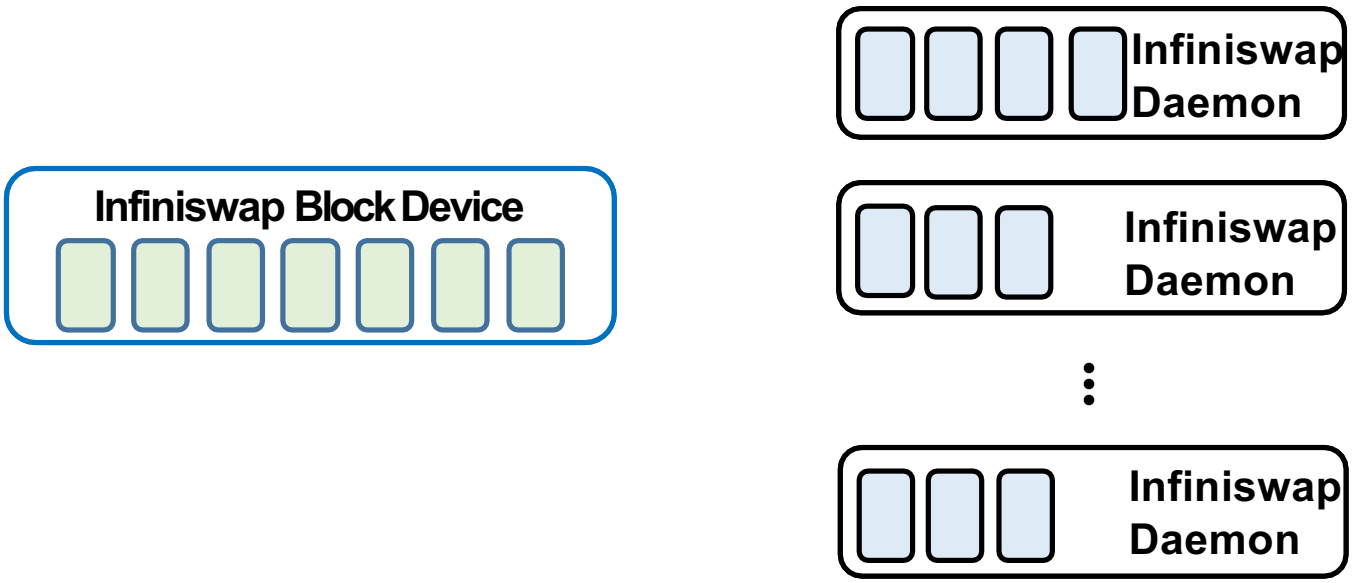
Management unit: memory page?



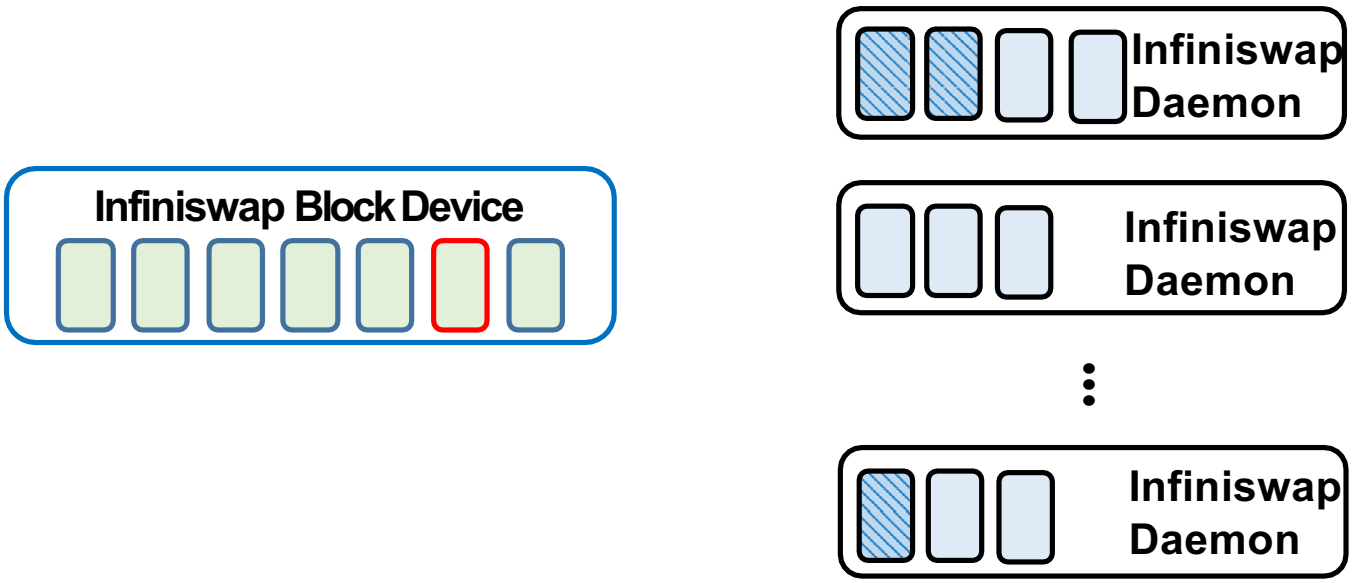
Management unit: memory slab!



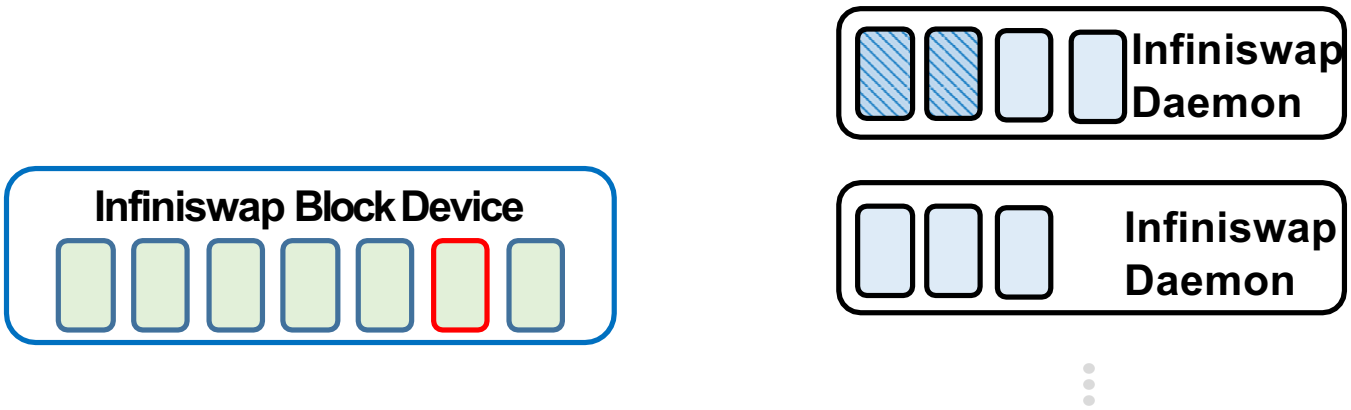
Management unit: memory slab!



Which remote machine should be selected?

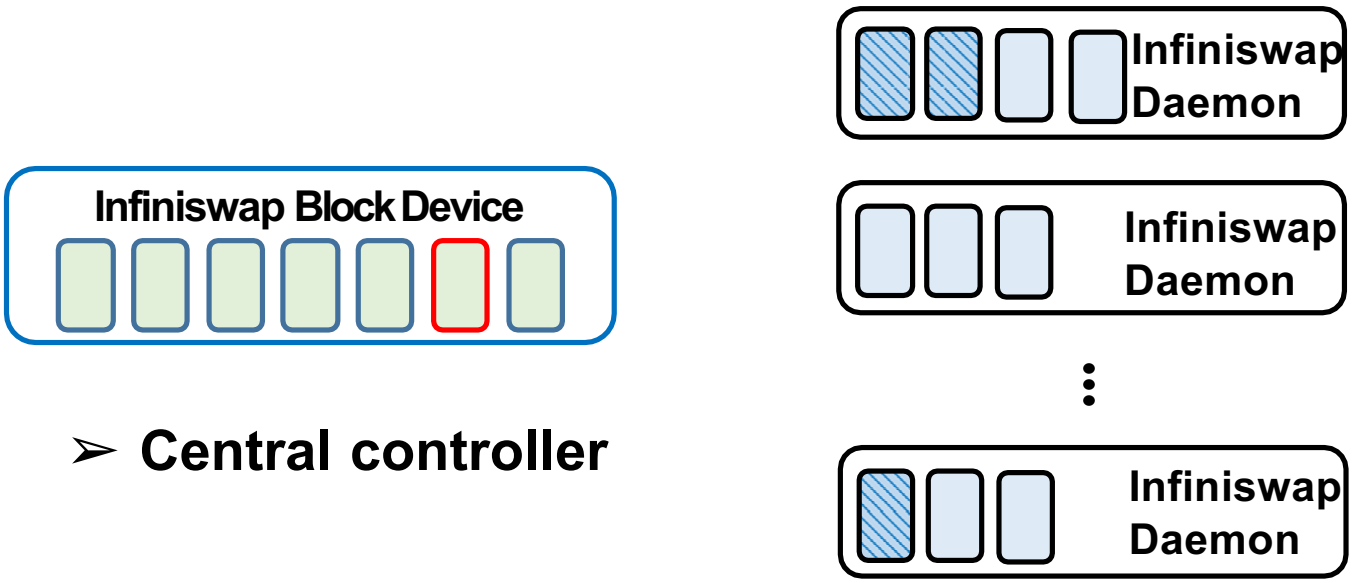


Which remote machine should be selected?



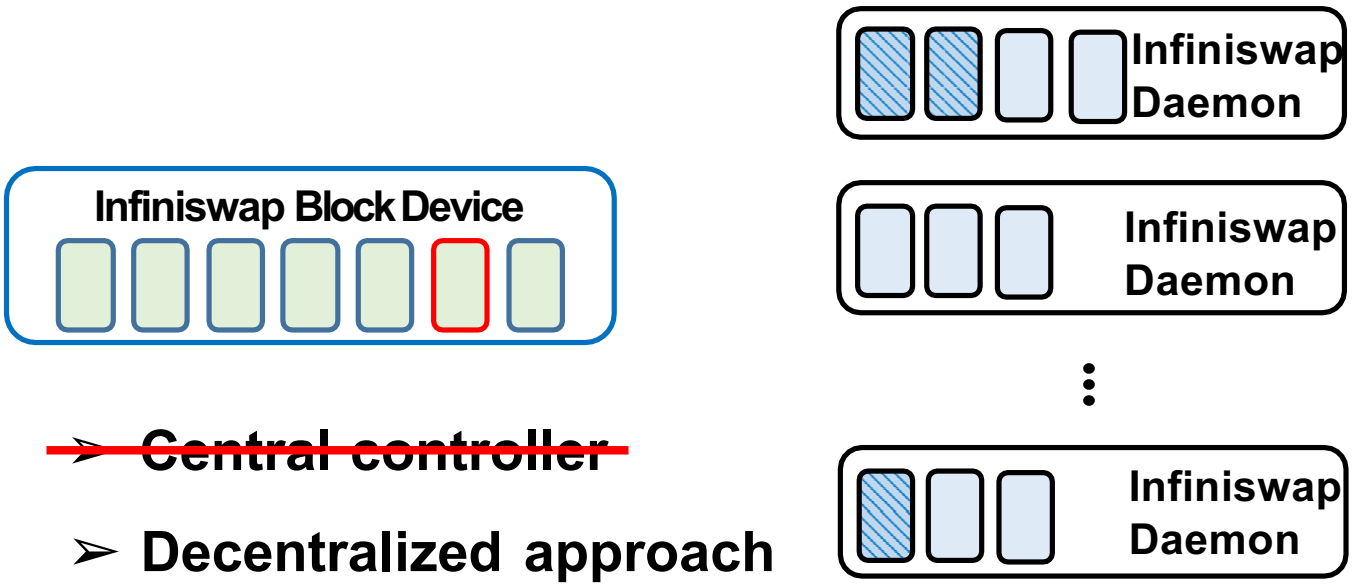
Goal: balance memory utilization

Which remote machine should be selected?



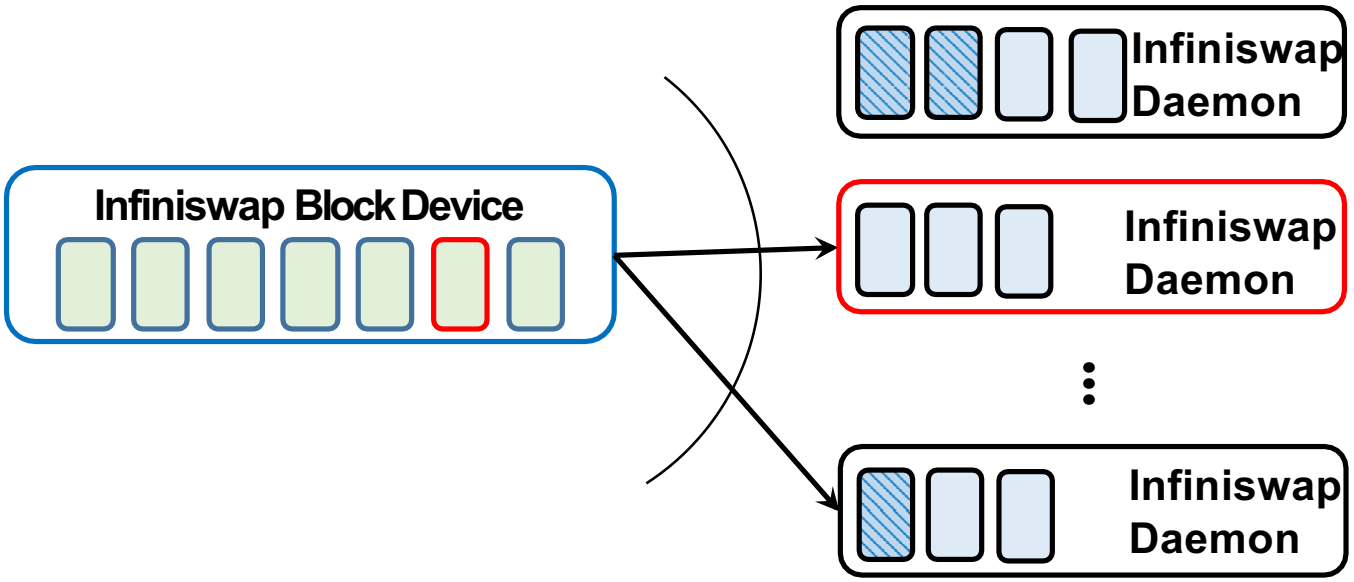
➢ Central controller

Which remote machine should be selected?



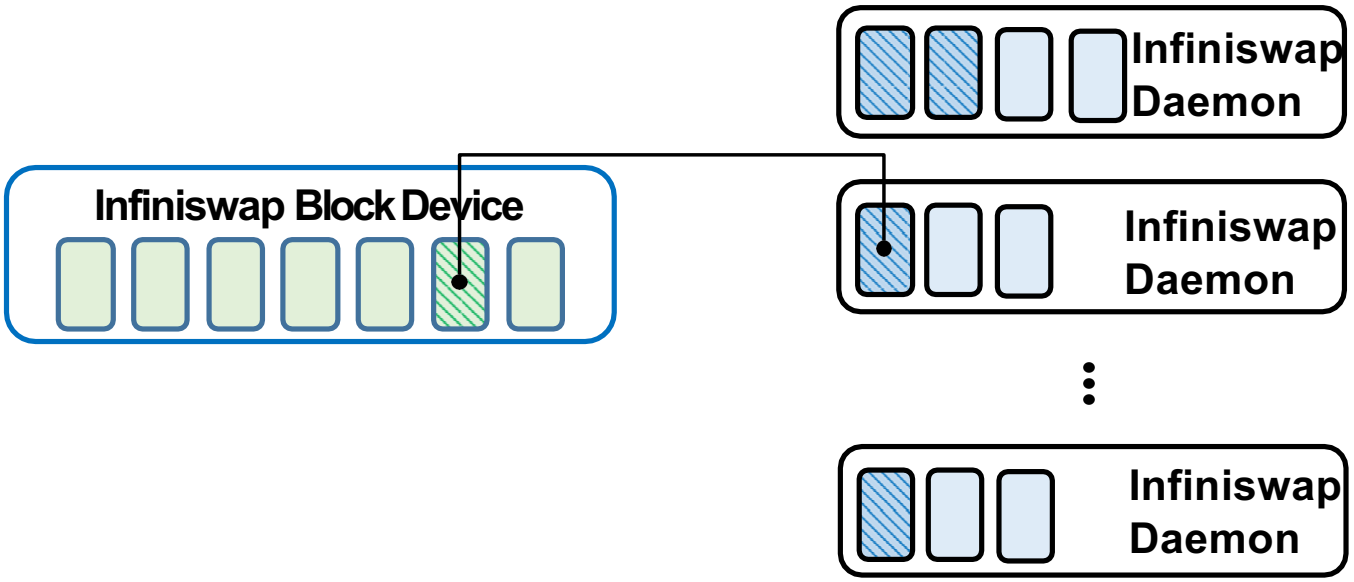
- ~~Central controller~~
- Decentralized approach

Power of two choices [1]



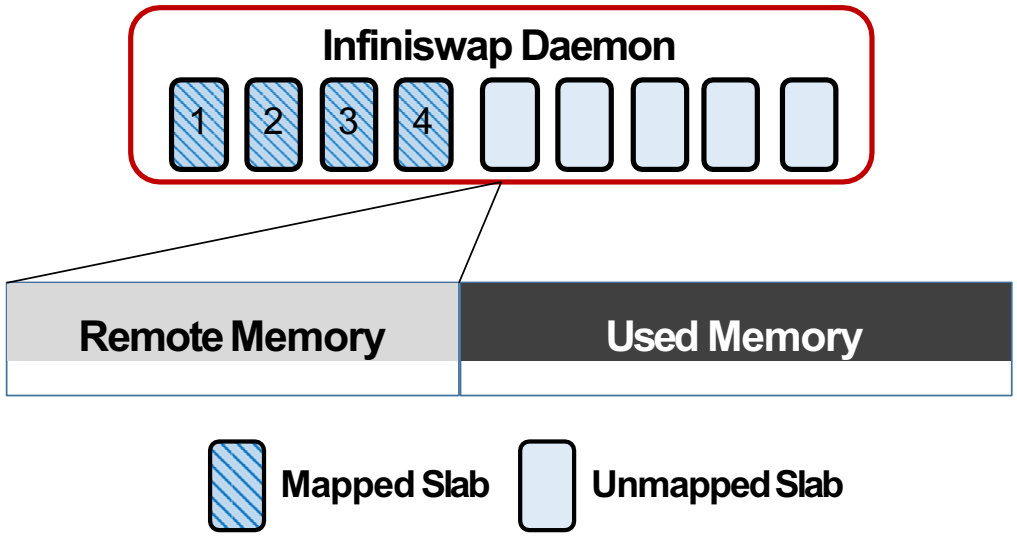
[1] Mitzenmacher, Michael. "The power of two choices in randomized load balancing.", Ph.D. thesis, U.C. Berkeley, 1996

Power of two choices [1]

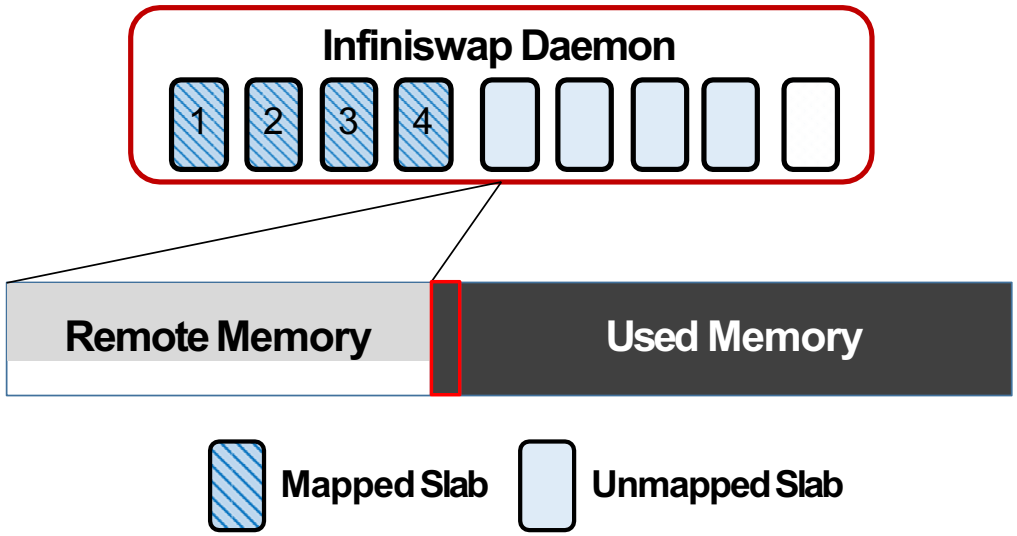


[1] Mitzenmacher, Michael. "The power of two choices in randomized load balancing.", Ph.D. thesis, U.C. Berkeley, 1996

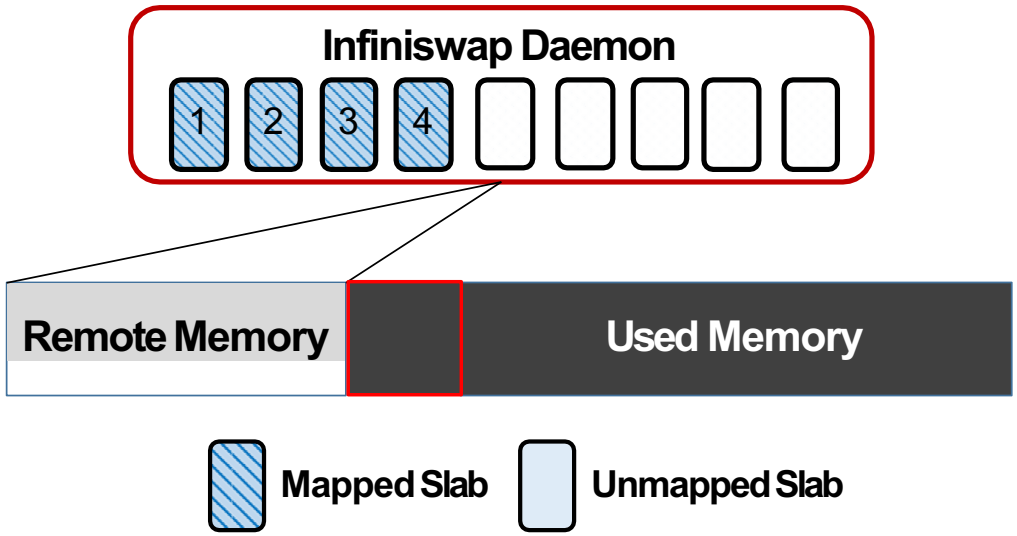
Slab eviction



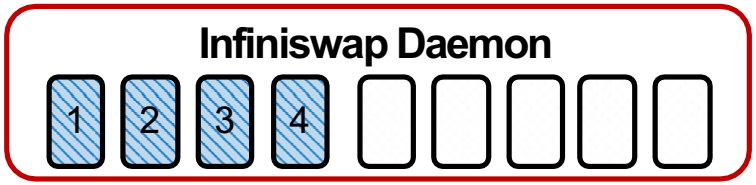
Slab eviction



Slab eviction

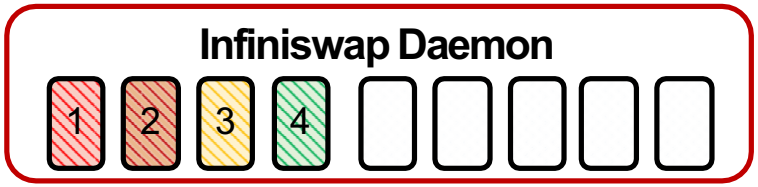


Which slab should be evicted?



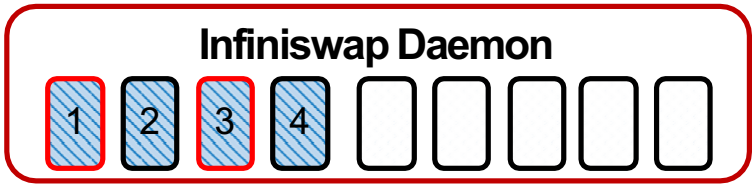
Daemon: Does not know the swap activities

Which slab should be evicted?



Daemon: Too expensive to query all the slabs

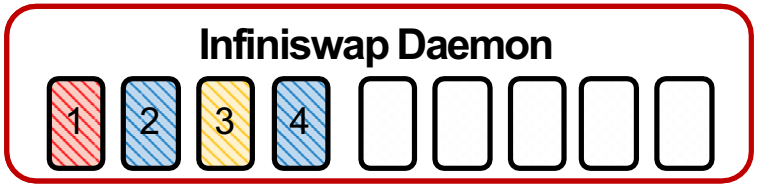
Power of multiple choices[1]



Select E least-active slabs from E+E' random slabs

[1] Park, Gahyun. "A generalization of multiple choice balls-into-bins." PODC'11

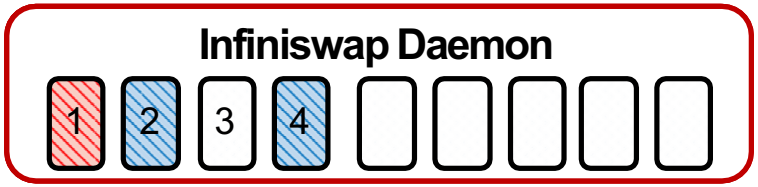
Power of multiple choices[1]



Select E least-active slabs from E+E' random slabs

[1] Park, Gahyun. "A generalization of multiple choice balls-into-bins." PODC'11

Power of multiple choices[1]



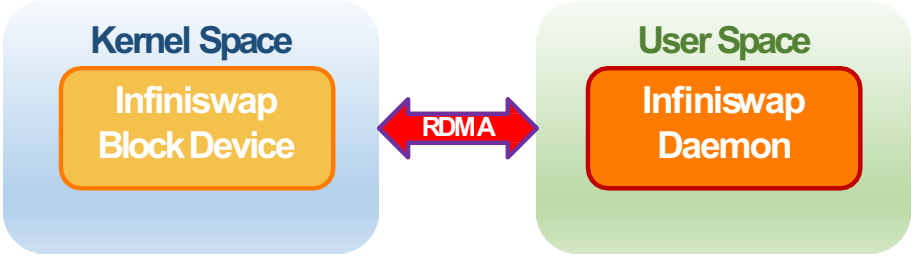
Select E least-active slabs from E+E' random slabs

[1] Park, Gahyun. "A generalization of multiple choice balls-into-bins." PODC'11

Agenda

- Motivation and related work
- Design and system overview
- **Implementation and evaluation**
- Future work and conclusion

Implementation



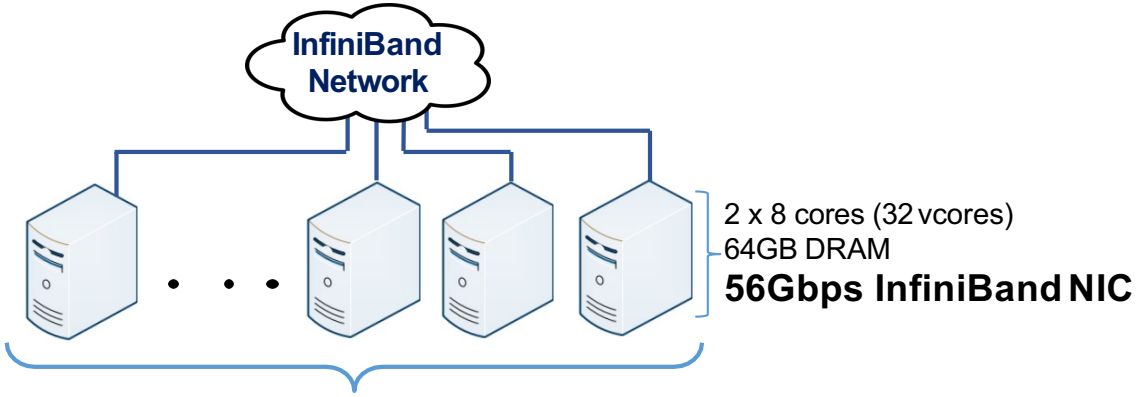
- **Connection Management**
  - **One** RDMA connection per active block device - daemon pair
- **Control Plane**
  - **SEND, RECV**
- **Data Plane**
  - **One-sided RDMA READ, WRITE**

What are we expecting from Infiniswap?

- **Application performance**
- **Cluster memory utilization**
- Network usage
- Eviction overhead
- Fault-tolerance overhead
- Performance as a block device

⋮

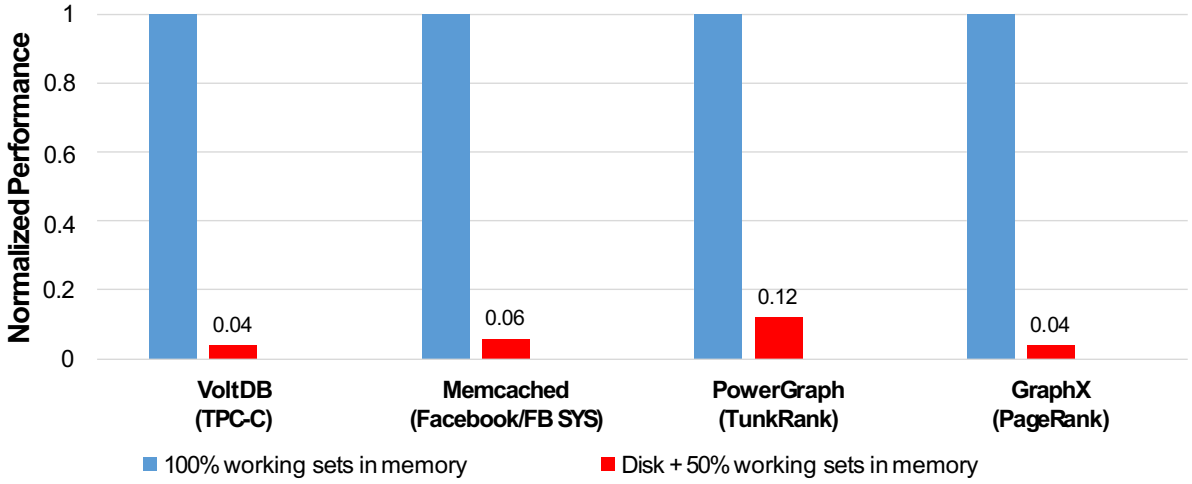
Evaluation



VOLTDB memCached powergraph GraphX

Application performance

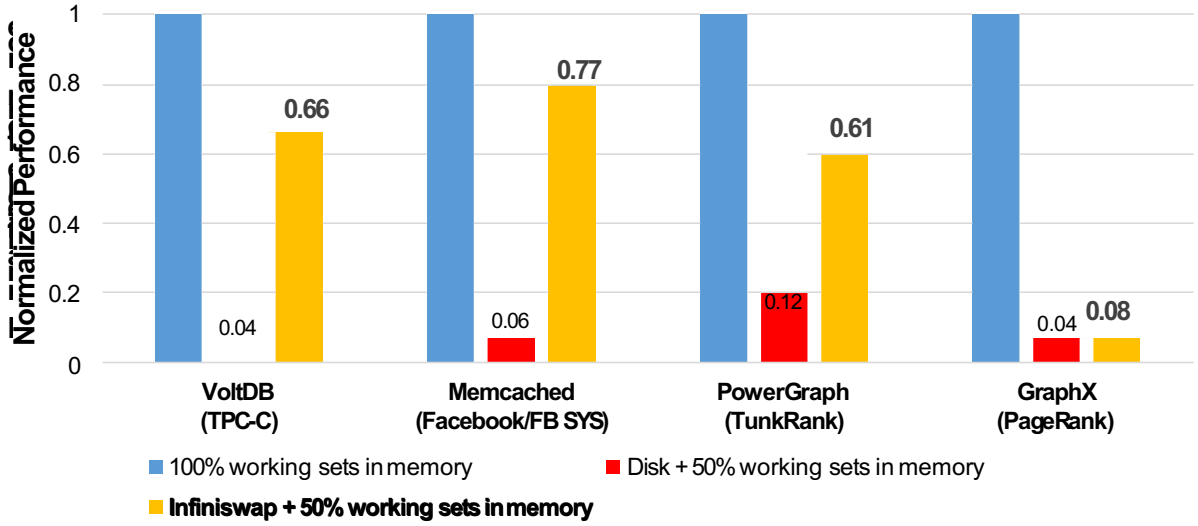
- 50% working sets in memory



- Application performance is improved by 2-16x

Application performance

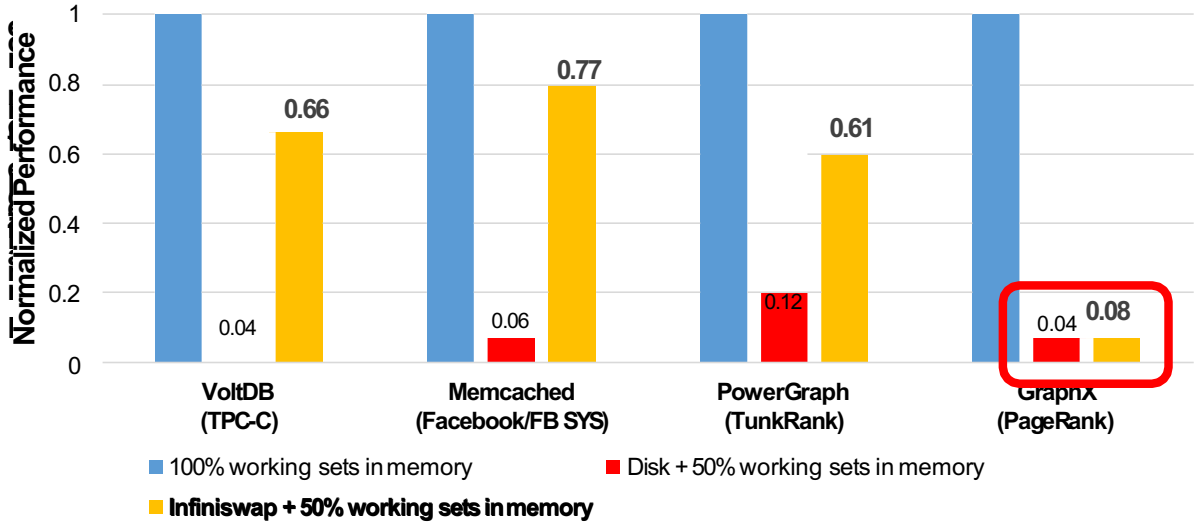
- 50% working sets in memory



- Application performance is improved by 2-16x

Application performance

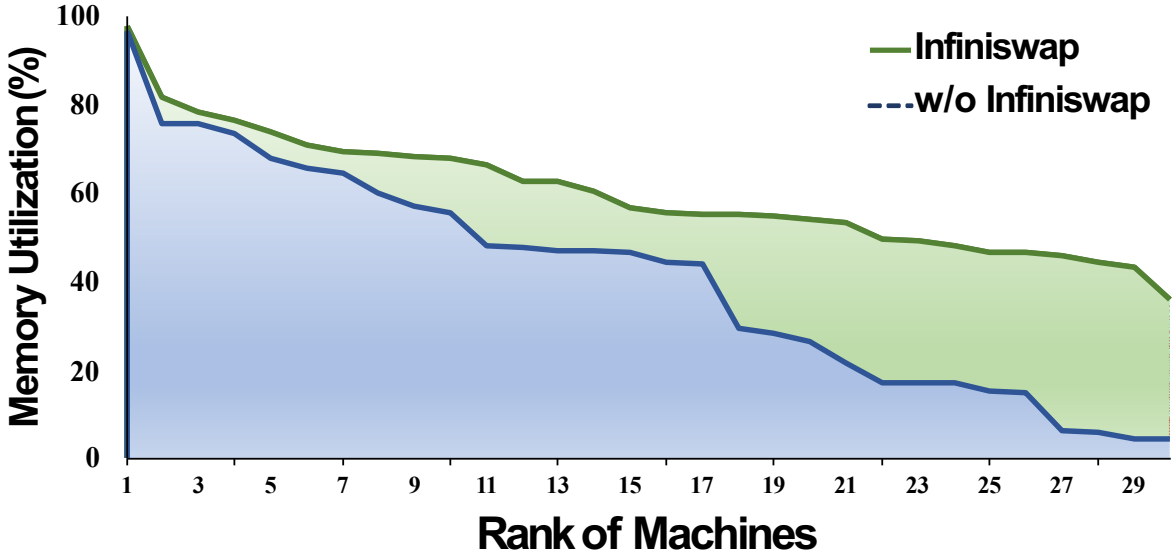
- 50% working sets in memory



- Application performance is improved by 2-16x

Cluster memory utilization

- 90 containers (applications), mixing all applications and memory constraints.



- Cluster memory utilization is improved from **40.8%** to **60%** (1.47x)

Agenda

- Motivation and related work
- Design and system overview
- Implementation and evaluation
- **Future work and conclusion**

Limitations and future work

- **Trade-off in fault-tolerance**
  - Local disk is the bottleneck
  - Multiple remote replicas
    - Fault-tolerance vs. space-efficiency
- **Performance isolation among applications**
  - W/o limitation on each application's usage
  - W/o mapping between remote memory and applications

Thanks